

Quantum / qai — Full Design Document

A programming language (**Quantum**, compiles to C, built in Rust) with a built-in AI development framework (**qai**). Core philosophy: **AI development should be simple, fast, and never a black box**. Nothing is hardcoded — every technique, layer, bridge, and visualization is an open, overridable interface.

1. Layers of AI Development (Pipeline)

- **Data layer** — collection, cleaning, labeling, streaming
 - **Model layer** — architecture (techniques, layers)
 - **Training layer** — the actual learning process
 - **Reasoning/inference layer** — test-time compute, chain-of-thought
 - **Agent layer** — tool-calling, planning, multi-step tasks
 - **Evaluation layer** — accuracy, F1, RMSE, benchmarks
 - **Deployment/serving layer** — APIs, edge/server targets
 - **Monitoring/MLOps layer** — drift detection, versioning
 - **Application layer** — chat interfaces, end-user products
-

2. Model Types / Techniques to Support

Classical ML: Linear/Logistic Regression, Decision Trees, Random Forests, Gradient Boosting (XGBoost/LightGBM), SVM, K-Means, Naive Bayes

Deep Learning: CNN, RNN/LSTM/GRU, Transformer, MoE (Mixture of Experts), GAN, Autoencoder/VAE, Diffusion, GNN, Reinforcement Learning (Q-learning, PPO, RLHF), Multimodal models

2026 landscape notes:

- MoE is now the dominant architecture for frontier models (GPT-5, Claude Opus, Gemini Ultra, DeepSeek, Qwen, GLM, Kimi all use it)

- Test-time compute / reasoning models are a new scaling axis alongside parameter count
 - Agentic AI (tool use, autonomous multi-step tasks) is the biggest usage shift, though production adoption is still low
 - Multimodal generation (image/video/audio) matured fast (Sora 2, Veo 3.1, Nano Banana Pro)
 - Physical AI / robotics moving from research to real deployment
 - Open-weight models are closing the gap with closed/proprietary ones
-

3. Core Language/Framework Philosophy

- **Nothing hardcoded** — every technique, layer, bridge, optimizer, and visualization is defined via an open interface, not baked into the compiler
- **Interpretability by default** — every model type supports inspection/explanation out of the box; "black box" is opt-in, not the default
- **Fewer lines than Python** — no heavy import chains; common techniques/layers available with zero imports
- **Compiled to C via Rust** — fast, with compile-time shape/architecture checking

The Technique Interface

```
technique Technique {  
    fn forward(input) -> output  
    fn objective(output, target) -> Signal // generic:  
    loss, reward, distance — technique decides  
    fn update(signal)  
    fn dashboard() -> DashboardLayout // technique  
    declares its own visualization  
}
```

Built-ins (Regression, CNN, Transformer, MoE, Diffusion, etc.) are just pre-shipped implementations of this same interface — no special compiler treatment. Users write their own the same way.

```
technique MyNewTechnique extends Regression {
  override fn objective(output, target) { /* custom math */
}
}
```

- `objective()` returns a generic `Signal` — not every technique uses "loss" (RL uses reward, diffusion uses denoising error, GANs use two competing losses, K-Means has no weights/loss at all)
- Same open-interface pattern applies to: **custom layers**, **custom bridges** (model merging), **custom visualizations**, **custom optimizers**

Fork-and-Protect Boilerplate System

```
qai new technique --from="Regression" --name="MyRegression"
qai new layer --from="Attention" --name="MyAttention"
qai new bridge --from="Adapter" --name="MyBridge"
```

Originals stay immutable in the core library; you always work on a named copy. Prevents accidentally breaking a working technique while still allowing full customization.

Formula Visualization

```
Regression.show_formula()           // renders  $y = mx + b$  with
live parameter values
MyRegression.diff(Regression)       // shows exactly what math
changed, readable notation
```

4. Interpretability & Black-Box-Breaking Features

Real-time thinking visibility:

- `watch()` — live activation/expert-routing streaming during training or inference
- Attention heatmap as a native output type on transformer-style blocks
- Decision trace / reasoning replay — scrollable log of intermediate reasoning steps with confidence at each point

- Expert routing visualizer for MoE — see which experts fire for which inputs

Visual learning dashboards (technique-aware, auto-generated):

- Regression → loss curve + residual plot; CNN → filter visualizations; Diffusion → denoising step preview; MoE → expert routing grid
- Gradient flow visualization (color-coded per layer: fast/slow/dying)
- Weight-change heatmaps over training time
- Data-to-decision lineage — click a prediction, see which training examples influenced it most

Black-box-breaking primitives:

- `.explain()` — human-readable rationale on any prediction, supported by every technique
- Confidence-annotated outputs — calibrated uncertainty, not just raw softmax
- `.what_if(input, change)` — counterfactual probing
- Failure-mode auto-detection — flags likely hallucination/overfitting/out-of-distribution inputs, like a compiler warning
- Training-time anomaly alerts — "loss diverging," "expert collapse detected," "gradient vanishing in layer 12" as first-class diagnostics
- Interactive replay mode — step through training epoch by epoch like a debugger

Weight interpretability & editing:

- Weight-to-concept translator — `model.weights.translate()` turns raw vectors into readable labels (e.g. "positive sentiment")
- Neuron/concept dictionary — browsable index of what each neuron/expert responds to
- Surgical editing — `model.remove()` / `model.patch()` to zero out or rewire specific neurons/experts directly
- Before/after diffing after an edit — auto re-run a test suite to confirm nothing broke
- Bias/error localization — trace a bad output backward to the responsible weights, like a stack trace

- Universal `.visualize()` — any model/layer/weight group renderable as graph, heatmap, 3D embedding space, tree, or flow diagram
- Embedding space explorer — walk visually through the model's internal concept space
- Version control for weights — git-like checkpoint/branch/merge/revert for model states
- Plain-language query interface — "why did you say X" answered by inspecting real weights/activations
- **Named brain regions** — developers name/scope sections of the network (e.g. "grammar block," "math block") so inspection and edits target names, not raw indices
- Built-in unlearning — forget specific data/behavior without full retraining

Note: guardrail visualization/management applies to guardrails the developer configures themselves — not a tool for stripping a model's underlying safety alignment.

5. Prediction Tracking & Fine-Tuning

```
model.predict(input).show_choices()  // top-5 candidates
considered + confidence + why the winner was picked
```

- Decision ledger — every inference logs its full candidate list, queryable later
- Choice diffing — compare decisions at checkpoint A vs B for the same input

```
model.finetune(
    data: my_dataset,
    freeze: ["layer1", "layer2"],  // freeze by name
    epochs: 5,
    watch: true
)
```

Fine-tuning modes:

- **full** — retrain everything (most power, most risk of overwriting old skills)
- **adapter** — small add-on layers learn new skill, base frozen (safe, cheap, reversible)
- **head-only** — retrain just the final output layer
- **lora** — lightweight low-rank adaptation
- **prompt-tuning** — learn an optimized prefix, no weight changes
- **distillation** — student model learns from a teacher model's outputs
- Fine-tune diff report — auto-summary of behavior change vs the base model
- **Fine-tuning individual attached skills** (see Skills section) without touching the base model or other skills

6. Layer Design & Training Control

```
model.add_layer("attention", after: "layer3", heads: 8)
model.remove_layer("layer7")
model.inspect("layer3").visualize()
```

- Drag-and-place visual layer editor (nodes you add/remove/rewire, generates code)
- Live epoch dashboard by default — loss, accuracy, gradient health, routing, all live, no external tools
- Pause/resume/rewind training — step epoch by epoch, branch, or roll back
- Architecture templates with override — start from a known-good template, only touch what you want to change
- Training playback — scrub a timeline slider through a full training run
- Per-epoch anomaly alerts ("expert 4 stopped activating," "gradients spiked in layer 6")
- Model comparison mode — diff architecture, weights, and predictions of two versions side by side

How layers actually work: usually one unified model where each layer learns a different *level of abstraction* (not a different subject) — trained together via backpropagation, communicating by passing tensors forward. MoE is the exception: different experts *within* a layer specialize on different sub-patterns via routing. Hybrid architectures (e.g. CNN → transformer → symbolic rules) are also supported — deliberate, named, manual composition.

7. Auto-Train to Target

```
model.autotrain(  
  target: "accuracy",  
  goal: 0.95,  
  max_epochs: 100,  
  strategy: "adjust_lr_and_layers"  
)
```

- Multi-target mode with weighted balancing (`target: ["accuracy", "loss"], weights: [0.7, 0.3]`)
 - Auto-stop on diminishing returns
 - Layer technique tagging (`layer.type("cnn")`) so mixed architectures stay self-documenting
 - Communication tap — `watch(layer3 -> layer4)` inspects tensors flowing between layers live
 - Lite architecture search — compiler suggests a starting architecture based on target metric + dataset
-

8. Mixing Techniques, Naming Layers, Merging Models

```
// Mixed/composed model, one architecture, named layers  
let model = qai.build()  
model.add_layer("vision", type: "cnn", name: "eyes")  
model.add_layer("reasoning", type: "transformer", name: "brain")
```

```

model.add_layer("rules", type: "symbolic", name:
"guardrails")
model.connect("eyes" -> "brain" -> "rules")

// Merging two separately pre-trained models
let hybrid = qai.merge(vision_model, language_model,
    connect_at: "vision.output -> language.input")

```

- Auto-compatibility checker — shape/dimension checks at compile time, not runtime crashes
- Named layer reuse across models — import a trained named layer into a different model
- Technique benchmarking mode — auto-train small versions of several techniques on the same data, report best fit
- Layer/technique marketplace/registry — publish and import components like a package manager

9. Full Model Lifecycle (Chainable Pipeline)

```

qai.build()
    .train()
    .test()
    .finetune()
    .merge()
    .benchmark()
    .visualize()
    .inspect()
    .edit()

```

10. Merging Models — Communication & Bridging

Two independently trained models don't share a representation space — some bridge is required, latency-free direct communication isn't realistic.

```

qai.merge(model_a, model_b, bridge: "adapter") //
lowest latency, small trained translator
qai.merge(model_a, model_b, bridge: "shared_embedding") //

```

```

fastest at inference, needs joint training
qai.merge(model_a, model_b, bridge: "router")           //
most flexible, more overhead

// User-selectable communication depth
qai.merge(model_a, model_b,
  communicate_via: "shared_embedding" | "adapter" |
"router" | "raw_output_only",
  frequency: "every_layer" | "final_output_only")

```

- Bridge auto-selection based on inspected input/output shapes
- Latency profiler for merges — exact overhead per inference call
- Cross-model attention — bridge as a small attention mechanism instead of pure translation
- Merge compatibility score — compile-time estimate of how well two models will work together

11. Serving, Visualization Extras, Lazy Loading, Export

```

let assistant = model.serve()           // auto-detects
chat vs classifier interface
assistant.chat("message", remember: true)

let model = qai.load("huge-model", lazy: true) // only
loads layers as they're actually called

model.export("model.gguf", quantize: "q4_0") // GGUF
export for edge/local inference

```

- Training visualization is technique-aware: regression shows loss/residuals/ R^2 , CNN shows filter maps + confusion matrix, automatically
- Session memory toggle for chat interfaces
- Layer-level lazy loading (especially valuable for MoE — only load routed experts)
- Auto quantization recommendation before export

12. Data Pipeline & Deployment Gaps

```
let data =
  qai.dataset.load("imdb_reviews").clean().split(train: 0.8,
  val: 0.1, test: 0.1)

let streamed =
  qai.dataset.stream("huggingface://imdb_reviews", batch_size:
  32)
    .clean().split(train: 0.8, test: 0.2)
let repo_data =
  qai.dataset.stream("github://user/repo/data.csv")
let kaggle_data = qai.dataset.stream("kaggle://dataset-name")
```

- Dataset streaming from Hugging Face / GitHub / Kaggle — avoids full downloads
- `model.deploy(target: "edge" | "server" | "mobile")`
- Versioning/rollback — `model.checkpoint("v1")`,
`model.rollback("v1")`
- Live A/B testing — `qai.compare_live(model_a, model_b,`
`traffic_split: 0.5)`
- Cost/compute estimator before training — `model.estimate_cost()`
- Dataset versioning tied to model checkpoints for reproducibility

13. Worked Example — Sentiment Classifier (Full Pipeline)

```
let data =
  qai.dataset.load("imdb_reviews").clean().split(train: 0.8,
  val: 0.1, test: 0.1)

let model = qai.build("transformer")
model.add_layer("embedding", size: 512)
model.add_layer("attention", heads: 8, name: "understanding")
model.add_layer("classifier", output: 2, name: "decision")

model.train(data: data.train, epochs: 10, target: "accuracy",
```

```
goal: 0.90, visualize: true)

let results = model.test(data.test)
model.predict("The movie was boring").explain()

let assistant = model.serve()
assistant.chat("Is this review positive: 'I loved every
second'")
```

14. Multi-Role Self-Play / Co-Training

For tasks like "one sub-model asks coding questions, another answers and improves" — combined model, user-defined roles, not hardcoded names:

```
let model = qai.build("combined")
model.add_role("asker", type: "transformer", goal:
"generate_questions")
model.add_role("answerer", type: "transformer", goal:
"solve_task")

model.connect("asker" -> "answerer", channel: "question")
model.connect("answerer" -> "asker", channel: "feedback")

model.set_evaluator(fn(question, answer) {
  return run_code_test(answer) // custom reward signal
})

model.train(mode: "self_play", rounds: 1000, task_domain:
"coding", visualize: true)
```

- Any number of roles, not just two (e.g. add a "critic" or "teacher" role)
- Role skill dashboard — compare each role's performance over rounds
- Difficulty auto-balancing so questions stay productively hard
- Swap/freeze individual roles mid-training
- Role template library (asker/answerer, teacher/student, generator/critic)

15. Output Control

```
model.serve(
  max_tokens: 200,
  stop_words: ["END", "\n\n"],
  required_words: ["def", "return"],
  banned_words: ["TODO", "pass"],
  temperature: 0.7
)
```

- Per-role output limits when using multi-role models

16. Custom Reward/Penalty Systems

```
model.set_reward(fn(output, target) {
  let base = accuracy(output, target)
  let penalty = if output.length > 100 { -0.2 } else { 0 }
  let bonus = if uses_keyword(output, "efficient") { 0.1 }
  else { 0 }
  return base + penalty + bonus
})
```

Fully custom scoring, not limited to standard loss/accuracy — connects directly to self-play evaluators.

17. Extensibility Registry & Guardrails

- Technique/layer/bridge registry — shareable, importable community components
- Technique composition — blend two techniques into a new one
- Auto-validation for custom techniques — sanity-check shapes/gradients before wasting compute
- Technique versioning

```
model.set_guardrails(
  defaults: true, // sane baseline if
  developer sets nothing
  block_topics: ["harassment"],
  max_output_length: 500,
```

```
    require_review: fn(output) { return contains_pii(output)
  }
)
model.guardrails.list()
model.guardrails.remove("topic_x")      // only for
guardrails the developer configured
```

18. Hardware Awareness & Performance

```
let hw = qai.detect_hardware() // { gpu, cpu_cores, ram,
vram }
model.train(auto_adapt: true)  // adjusts batch size /
precision / model size to available hardware
```

- `Tensor` as a first-class language type (not a library import) — operations auto-parallelize (SIMD/GPU), no manual loops
- Compiler warning if a manual loop over a `Tensor` could be vectorized instead
- Runs down to CPU-only/low-RAM systems with automatically smaller architectures

19. Serving Infrastructure

```
qai.serve(model, port: 8080, protocol: "rest" | "grpc" |
"websocket", auth: "api_key", rate_limit: 100/min)
qai.dashboard(port: 3000) // auto-generated local GUI from
the model's own introspection data
```

- Logging/telemetry — every request/response logged with latency + confidence
 - Model registry — versioned, searchable catalog of trained models
 - Prompting interface helper for LLM-style models —
`qai.prompt("template {input}")`
-

20. Weight-to-Words & Educational Visualization

```
model.weights.translate() // "0.803, -0.001, 0.42..." →  
"strongly associates with 'positive sentiment'"
```

What a learner should be able to see, per concept:

- **Probability** — live bar chart of all candidate outputs and their scores, not just the winner
- **Regression** — literal scatter plot + fitted line redrawing itself each epoch, residual lines shrinking visibly
- **Classification** — 2D decision boundary redrawing live; confusion matrix filling in during training
- **Neural nets generally** — node/edge diagram with edges lighting up by weight magnitude; forward-pass animation showing values transforming layer by layer
- **Gradient descent** — a loss-landscape visual with a "ball rolling downhill," the physical intuition behind the math
- **Per-technique flagship visuals** — CNN: filter maps; RNN: unrolled timestep view; Transformer: attention heatmaps; MoE: expert routing grid; GAN: generator/discriminator score tug-of-war; Diffusion: step-by-step denoising reveal

Dashboard design principle: each technique declares its own `dashboard()` (unique panels like scatter plots or denoising previews), but shares common building blocks (loss curve, gradient health) from a shared visual library. Techniques without weights (e.g. K-Means) simply omit weight-related panels.

Layer parameters explained (example):

- `embedding(size: 512)` — turns tokens into 512-number vectors the model can do math on
- `moe(experts: 32, active: 4)` — 32 specialized sub-networks exist, only 4 run per input (router-selected)
- `classifier(output: 3)` — produces 3 probability scores, one per category

Tokens: text is split into tokens (words/sub-words) before embedding —

```
qai.tokenize("hello world", method: "bpe").
```

21. Universal Help System

```
model.add_layer("moe", experts: 32).help()
qai.dataset.stream.help()
technique.help()
```

Every callable function/method/parameter explains itself — a language-level guarantee, not optional documentation. Ideally includes runnable examples, not just descriptions.

22. Multimodal Generation

```
let model = qai.build(technique: "diffusion")
model.train(data: image_dataset, target: "image")

let model3d = qai.build(technique: "diffusion", output_type:
"mesh")
let audio_model = qai.build(technique: "diffusion",
output_type: "waveform")

model.generate(prompt: "a low-poly
castle").export("castle.obj")
```

Combining modalities:

```
let model = qai.build()
model.add_layer("video", type: "diffusion", name: "visuals")
model.add_layer("audio", type: "diffusion", name: "sound")
model.connect("visuals" <-> "sound", bridge:
"shared_embedding")
```

Technique suggestion advisor (no silent magic):

```
qai.suggest_technique(data: my_dataset)
// explains its reasoning, doesn't auto-pick for the user
```

Long-form consistent video (technique-defined, not hardcoded globally):

```
technique VideoContinuation extends Diffusion {
  fn conditioning(previous_chunk) -> Signal {
    return extract_last_frames(previous_chunk)
  }
}
model.generate_long(technique: VideoContinuation,
  chunk_length: 30, total_length: 3600)
```

- Style/consistency score tracker between chunks
- Auto-stitching/blending at chunk boundaries

Targeted weakness fine-tuning:

```
let model = qai.load("video-gen-base")
model.finetune(data:
  qai.dataset.stream("huggingface://animation_clips"),
  mode: "adapter", focus: "animation_style")
```

23. Knowledge Distillation

```
let teacher = qai.load("large-model")
let student = qai.build("small-transformer")
student.distill_from(teacher, data: dataset, temperature:
  2.0)
```

24. Composable Skills System

```
let model = qai.load("3d-generator")

let rigging_skill = qai.skill.train(
  base: model,
```

```

    data:
qai.dataset.stream("huggingface://rigged_characters"),
    mode: "adapter",
    name: "rigging"
)

model.attach_skill(rigging_skill)
model.generate("a walking character").rig()

model.finetune_skill("rigging", data: more_data, epochs: 5)
// improve an attached skill directly

```

- Skill versioning (`rigging_skill.version("v2")`) and rollback
- Skill diffing between versions
- Skill detach — clean removal back to pre-attachment state
- Skill dependency graph (`skill.requires("rigging")`)
- Skill marketplace — publish/import trained skills across compatible models
- Skill combination — merge frequently-paired skills into one package
- Compatibility check before attaching
- Auto-benchmark after attach — catches skill attachment degrading old abilities (catastrophic forgetting)

25. Benchmark Integration

```

let report = qai.benchmark.submit(model, target:
"public_leaderboard")
report.qr_code()
report.live_view()
report.compare_to_last()

```

- Auto category detection based on model technique/output type
- Historical benchmark tracking per model

26. Additional Novel Ideas

- **Active learning** — model requests the specific unlabeled examples it's most uncertain about (`model.request_labels(n: 20)`)
- **Curriculum learning** — auto-order training data easy → hard
- **Reproducibility fingerprinting** — hash of data version + seed + technique + hyperparameters for exact re-runs
- **Model watermarking** — invisible signature in generated outputs for provenance tracing
- **Carbon/energy cost tracking** — visible compute/power cost per training run
- **Adversarial robustness probe** — `model.stress_test()` automatically searches for fragile edge cases
- **Continual learning safety net** — automatic memory buffer testing against old behavior when fine-tuning/attaching skills
- **Self-critique loop** — model reviews and revises its own output against a learned rubric before returning it
- **Uncertainty calibration checker** — verifies stated confidence actually matches real accuracy
- **Dataset bias scanner** — flags class imbalance/skew before training starts
- **Multi-developer live training sessions** — shared, watchable (or steerable) training runs
- **"Explain everything" report** — one command bundling architecture, data, guardrails, benchmarks, bias scan, and energy cost into one document
- **Model card auto-generation** — summary doc for sharing a finished model with others

27. Visual Design Direction (Dashboard)

A working mockup was built: dark, lab-instrument/oscilloscope aesthetic (not a generic SaaS dashboard) — monospace type, phosphor-green loss trace, amber accuracy trace, animated expert-routing grid, live gradient-health bars, live prediction/decision log, and a weight → concept translation panel. Each technique's dashboard is composed from shared panels (loss curve, gradient health) plus its own unique ones (scatter+line for regression, denoising preview for diffusion, routing grid for MoE).

28. Standard Accessors — Shared Across Every Technique

```
technique Technique {  
    // SHARED — every technique guarantees these, same names,  
    same shape  
    fn tokens() -> [Token]  
    fn weights() -> WeightSet  
    fn loss() -> f64           // generic Signal underneath  
    fn accuracy() -> f64  
    fn parameters() -> ParamSet  
    fn param_count() -> int  
  
    // UNIQUE — each technique adds its own on top (e.g.  
    Regression.formula(), Diffusion.denoise_steps())  
}
```

`model.weights()`, `model.loss()`, `model.accuracy()`, `model.tokens()`, `model.parameters()` work identically no matter the technique — one interface that transfers everywhere, with technique-specific extras layered on top, never replacing the shared baseline.

29. More Novel Interpretability & Debugging Ideas

- **Model X-ray** — `model.xray()` — single blueprint diagram: every layer, its parameter count, memory footprint, compute cost, at a glance
- **Sonification mode** — `model.listen()` — converts training signals (loss, gradient flow, expert activity) into real-time sound; pitch rises as loss drops, static/noise on instability — an accessibility feature and a genuinely different way to catch a bad training run
- **Dream/feature visualization** — `neuron.dream()` — generates a synthetic input that maximally activates a specific neuron/expert (DeepDream-style), showing what it "wants to see" instead of guessing from data
- **Model genealogy tree** — `model.genealogy()` — visual family tree of every ancestor (base model, fine-tunes, merges, skill attachments) that

produced this exact model

- **Per-token loss heatmap** — highlights which specific tokens a language model struggled with in a sentence, not just one aggregate loss number
- **Parameter sensitivity analysis** — `model.sensitivity()` — ranks which parameters actually influence output most, like feature importance but for weights, useful for knowing what's safe to prune/edit
- **Counterfactual training branches** — from any checkpoint, spin off parallel "what if I changed this hyperparameter" branches and compare, like git branches for experiments
- **Annotation layer on live graphs** — click a point on a training graph, leave a note, saved permanently with that run's history

30. Classic ML Workflow Essentials (Gaps Filled)

```
let ensemble = qai.ensemble([model_a, model_b, model_c],
method: "stacking") // or "bagging" / "boosting"

model.search_hyperparams(method: "bayesian", trials: 50,
target: "accuracy") // grid / random / bayesian

model.cross_validate(folds: 5)

model.add_layer("dense", dropout: 0.3, regularize: "l2") //
regularization: dropout, L1/L2, batch norm
```

- **Learning rate schedulers** — warmup, decay, cosine annealing — explicit and swappable, not just implicit inside `autotrain`
- **Early stopping** — halts training automatically on validation performance plateau/regression (overfitting-specific, distinct from the "diminishing returns" auto-stop)
- **Transfer learning** — an explicit named entry point on top of `finetune`, since "reuse a model trained on task A for task B" is such a common beginner pattern it deserves its own clear name/example in docs

More novel ideas:

- **Model "diet" mode** — `model.compress(target_size: "50%")` — auto-picks the best mix of pruning/quantization/distillation to hit a size

target with minimal accuracy loss

- **Training race mode** — run two hyperparameter configs side by side on a split dashboard, racing to see which converges faster
- **Failure replay library** — every bad failure on a live input auto-saves into a growing "hard examples" set for later retraining

31. Design Philosophy Check-In

The framework is **not hardcoded** — techniques, layers, bridges, optimizers, dashboards, and guardrails all route through the same open interface pattern (Section 3), so new capabilities (skills, self-play, multimodal) extend the system instead of requiring special-cased additions. This consistency — one pattern, applied everywhere — is what makes it "powerful by design" rather than just feature-rich.

Transformer clarification: Transformer is a built-in `Technique`, exactly like Regression, CNN, or MoE — not special-cased in the compiler:

```
let model = qai.build(technique: "transformer")
```

32. Historical / Foundational Techniques (for teaching AI history)

Runnable as real techniques with the same interface as modern ones — not just documentation — so a course can build a 1958 perceptron and a 2026 MoE transformer with the same three lines of code:

```
let model = qai.build(technique: "perceptron")
```

- **Perceptron** (1958) — original single-layer neural unit, ancestor of deep learning
- **Symbolic AI / expert systems** — rule-based "if-then" reasoning, pre-statistical-ML era
- **K-Nearest Neighbors (KNN)** — classify by proximity to labeled examples

- **Decision stumps / ID3 decision trees** — original tree-splitting algorithms, precursors to Random Forest/XGBoost
 - **Hidden Markov Models (HMM)** — sequence modeling before RNNs (original backbone of speech recognition)
 - **Bayesian Networks** — probabilistic graphical models for reasoning under uncertainty
 - **Genetic algorithms / evolutionary computation** — optimization via natural-selection-style search instead of gradients
 - **Self-Organizing Maps (Kohonen networks)** — early unsupervised clustering/visualization technique
 - **Hopfield Networks & Boltzmann Machines** — early recurrent/energy-based models, precursors to modern generative models
-

33. Security Module

```
// Data poisoning defense
let data = qai.dataset.stream("huggingface://some_dataset")
  .security_scan() // flags outlier clusters, label-
                  // flipping patterns, duplicated adversarial samples
  .clean()

// Prompt injection defense (served/agent models)
qai.serve(model,
  input_filter: "prompt_injection_guard",
  sandbox_tools: true)
```

- **Dataset poisoning scanner** — `data.security_scan()` catches poisoned data before training starts
- **Prompt injection guard** — built-in classifier screening incoming requests before they reach the model
- **Tool-call sandboxing** — agentic tool calls run isolated by default; no unchecked filesystem/network access
- **Supply-chain verification** — marketplace imports (techniques/skills/layers) are checksummed/signed; `qai.import()` warns on signature mismatch

- **Model weight tamper detection** — `model.verify_integrity()` hash-checks weights against a known-good fingerprint
- **Differential privacy training** — `model.train(privacy: "differential", epsilon: 1.0)` adds calibrated noise so training data can't be leaked
- **Output leakage filter** — detects verbatim training-data reproduction or PII in outputs before serving
- **Standing automated red-teaming** — `stress_test()` extended to run periodically against a live served model, not just once pre-deployment

34. Execution-Grounded Learning (VM/Environment Connection)

```
let vm = qai.connect_vm(sandboxed: true, language: "python")

model.train(
  mode: "execution_grounded",
  environment: vm,
  task: "write a function that sorts a list",
  evaluator: fn(code) {
    let result = vm.run(code)
    return result.tests_passed && result.no_errors
  }
)
```

A model learns from real execution feedback (does the code actually run/pass tests?), not just pattern similarity — a genuine RL step beyond imitation learning, and it plugs directly into the self-play evaluator system (Section 14).

Environment — a pluggable interface, same pattern as **Technique** :

```
environment Environment {
  fn run(action) -> Result
  fn reset()
  fn observe() -> State
}
```

Built-ins: VM/sandbox (code), browser (web navigation), game engine (classic RL), robotic/physics simulator, spreadsheet/document environment. All models train against any of these with the same `mode: "execution_grounded"` call.

- **Execution trace as training signal** — feed the model the actual runtime error/trace on failure, not just pass/fail
- **Difficulty-graduated coding curriculum** — auto-escalate task complexity as pass rate improves
- **Multi-language VM pool** — `qai.connect_vm(languages: ["python", "rust", "c"])`
- Sandboxing here is the default safety posture, tying directly into the Security Module above

35. Input/Output Pipeline & Multimodal Security

Receiving information — per-modality sanitization, not just text:

```
model.input_channels(["text", "image", "audio", "video",  
"file"])  
let clean_image = qai.scan_multimodal(image, checks:  
["steganography", "hidden_text", "adversarial_perturbation"])
```

- Text → prompt injection guard
- Images/video → steganography, hidden OCR-readable text, adversarial pixel perturbations (checked per-frame for video)
- Audio → inaudible/ultrasonic embedded commands, adversarial perturbations
- Files → hidden text layers, embedded scripts/macros, metadata injection

Where output gets diverted — and how to close it:

- **Steganographic exfiltration defense** — `output.leak_scan()` checks generated artifacts for abnormal noise patterns hiding smuggled data
- **Agentic egress control** — `model.set_egress(allowed_destinations: [...])` allowlists where actions/output can actually go, preventing a

manipulated agent from being tricked into leaking data via a legitimate-looking tool call

- **Output volume anomaly detection** — flags abnormal output volume/frequency as a sign of abuse, behavior-based rather than just a hard rate limit

More ideas:

- Cross-modal consistency check — verify paired outputs (e.g. caption + image) actually agree before releasing
- Provenance watermarking by default across all output modalities, not just images
- Modality-specific stress testing — `stress_test()` extended to adversarial images/audio/video, not just text

36. Output Renderer System (Capability-Checked)

```
let model = qai.build(technique: "diffusion", output_type:
"video")
model.capabilities()           // → ["video"]
model.generate(prompt).render_as("video") // ✅ matches
capability
model.generate(prompt).render_as("3d")    // ❌ clear
error, no silent failure/substitution
```

```
renderer OutputRenderer {
  fn required_capability() -> String | [String] //
renderers can require multiple capabilities
  fn render(model_output) -> Artifact
}
```

- **Two-way capability matching** — renderers declare what they need, models declare what they have; `render_as()` checks both before running
- **Multi-capability models** — a model can genuinely support more than one output type if actually built/trained for it
- **Capability certification** — `model.certify_capability("video")` test-generates and validates a real sample, not just a claimed label

- **Composite renderers** — e.g. video-with-audio requires `["video", "audio"]` present together
- **Closest-match suggestion on mismatch** — error suggests the nearest real option instead of silently substituting
- **Custom renderer boilerplate** — `qai new renderer --from="ImageRenderer" --name="MyRenderer"`, same fork-and-protect pattern as techniques
- **Renderer versioning** for backward compatibility as output formats evolve

37. Model-to-Model Piping & Full Modality List

```
let model_a = qai.build(technique: "transformer")
let model_b = qai.build(technique: "diffusion", output_type:
"image")

model_a.generate("describe a
scene").pipe_to(model_b).render_as("image")
```

`.pipe_to()` reuses the merge/bridge system inline, with the same compatibility check as `render_as()` — any number of models can chain (text → image → video → 3D), or loop back into themselves for iterative refinement.

Native modality list: Text, Image, Video, Audio, 3D (mesh/point cloud), Documents (PDF/DOCX/spreadsheets), Tabular/structured data, Time-series data, Graph/network data, Action sequences (robot/game control), raw Embeddings/vectors — plus fully custom renderer types (e.g. legal contracts, CAD files, sheet music) via the same open `OutputRenderer` interface, not limited to this built-in list.

Blind spots identified:

- **Streaming/partial output** — `model.generate_stream(prompt, on_chunk: fn(piece) {...})` needed for token-by-token or frame-by-frame generation
- **Pipeline cycle safety** — feedback loops between chained models need a default `max_iterations` /cycle-detection guard

- **Cross-model format mismatches** — finer than capability matching (e.g. 512px vs 1024px images) needs lightweight auto-adapters at the pipe level
- **Whole-pipeline tracing** — `pipeline.trace()` to show input/output at every hop across a multi-model chain, since per-model `.explain()` / `.watch()` don't cover cross-model chains
- **Pipeline cost/latency estimation** — `pipeline.estimate_cost()` before running a long chain
- **Backpressure handling** — defined behavior (queue/buffer/block) when one model in a chain is much slower than others

38. Multi-Output Generation & Model Piping

Native joint output — one model, multiple output heads, generated from the same forward pass (contextually coherent by construction):

```
let model = qai.build(technique: "transformer")
model.add_layer("text_head", type: "classifier")
model.add_layer("image_head", type: "diffusion")
let result = model.generate(prompt).render_as(["text",
"image"])
result.text
result.image
```

Piped/orchestrated output — separate specialized models chained together (how most real production multimodal systems actually work today — a language model calls out to an image model as a tool, rather than one model natively producing everything):

```
model_a.generate(prompt).pipe_to(model_b).pipe_to(model_c).render_as(["text", "code", "image"])
```

Orchestration is cheaper/faster to build; native joint generation requires training on paired multimodal data from the start and is architecturally "more correct" but rarer. Quantum should support both explicitly.

Bundle type — keeps multiple typed outputs together with linking metadata instead of loose return values:

```
type Bundle {
  text: Text?
  image: Image?
  code: Code?
  video: Video?
}
```

39. Additional Blind Spots

- **Emergency stop / kill switch** — `model.emergency_stop()` — a hard circuit breaker to instantly halt a running agentic model mid-action, distinct from guardrails (which prevent bad behavior in advance)
- **Language tooling for Quantum itself** — syntax highlighting, autocomplete, inline error checking (a language server) — developer experience of the language matters as much as the framework's AI capabilities
- **Subgroup/fairness evaluation** — `model.evaluate_by_group(feature: "region")` — catches a model that's strong overall but weak for a specific data slice, which pre-training bias scanning won't catch
- **Auto-generated documentation** — `qai.generate_docs()` compiles every `.help()` entry into a real docs site/file automatically
- **Semantic versioning for the language/framework** — a clear breaking-vs-non-breaking compatibility contract as Quantum evolves

40. Low-Memory Inference & Lazy Loading

Modeled after real tools like **AirLLM** (open-source, streams model layers/experts instead of loading the whole model into memory, letting 70B+ models run on 4GB VRAM without quantization) and **llama.cpp/GGUF** (CPU-friendly quantized inference), **vLLM** (memory-efficient serving), **ONNX Runtime/ExecuTorch** (cross-platform/mobile), **Ollama** (one-command local run UX).

```
let model = qai.load("huge-model",
  strategy: "layer_stream", // one layer resident in
```

```

memory at a time, unload after use
    prefetch: true)          // next layer loads while
current one computes, hides latency

model.load(auto_adapt: true)    // picks full-load / layer-
streaming / quantized-load automatically
                                // based on
qai.detect_hardware()

```

- Disk-as-extended-memory fallback when RAM/VRAM is insufficient
- `model.load_report()` — shows expected speed tradeoff of layer-streaming vs full load before committing
- Partial GPU/CPU offload — split a model across both simultaneously rather than all-or-nothing

41. Model Memory System

Memory means different things in real AI systems, and Quantum should map onto the real distinction rather than a single generic idea:

```

model.memory.weights          // baked-in, learned via training
- slow/permanent, changes only via finetune
model.memory.working          // current context window - fast,
temporary, per-session
model.memory.retrieval        // external store (vector-DB
style) - fast to update, no retraining required
    .add("fact")
    .search(query)

```

- **Weights** = long-term but static memory (what training compressed in)
- **Working memory** = the context window, cleared per session unless explicitly carried forward
- **Retrieval memory** = how real production systems give a model fast-changing "long-term memory" without retraining (RAG-style) — this is the actual answer to "can a model use storage as memory"

More memory ideas:

- Memory conflict resolution — `model.memory.retrieval.resolve_conflicts()` flags contradicting stored facts
- Memory expiry/TTL — `add("fact", expires: "30d")` for time-sensitive info
- Auto memory-strategy picker — decides whether new info belongs in retrieval (fast-changing) vs triggers a fine-tune (stable skills)
- Knowledge freshness report — `model.memory.staleness_report()` flags likely-outdated trained-in facts

42. Stateful Sessions, Auto-Checkpointing & Declarative Syntax

Fixes the "running the script again creates a brand-new model" problem, database-style:

```
let model = qai.build("regression", name: "my_model",
if_exists: "resume")
// first run: creates fresh. every run after: resumes saved
progress instead of starting over

model.train(data, autosave: true, autosave_interval:
"every_epoch")
```

- `if_exists: "resume"` (default-safe, idempotent, like `CREATE TABLE IF NOT EXISTS`) vs `if_exists: "fresh"` when a clean model is actually wanted
- Autosave on interval and on shutdown, so nothing is lost without an explicit save

Optional SQL-style declarative syntax — alternative front-end syntax, selected via a file directive, compiling down to the exact same underlying model definition as normal Quantum code — lets database-minded beginners and programmers both build the same thing in the syntax that matches how they think:

```
#style: declarative

CREATE MODEL sentiment_classifier
USING transformer
WITH embedding(size: 512), attention(heads: 8),
classifier(output: 2)

TRAIN sentiment_classifier ON imdb_reviews UNTIL accuracy >
0.90

SELECT prediction, confidence FROM sentiment_classifier WHERE
input = "great movie"
```

43. Abilities, Knowledge Gaps & Targeted Training

Hierarchical abilities, drillable into sub-areas:

```
model.abilities() // → {
  code_generation: 0.81, reasoning: 0.74, ... }
model.abilities().drill_down("code_generation") // → {
  security: 0.42, web_dev: 0.85, ... }
model.finetune(data: security_data, focus:
"code_generation.security")
```

Auto-diagnose + auto-improve loop, tied to the benchmark system:

```
let diagnosis = model.auto_diagnose() // →
weakest area found automatically
model.auto_improve(diagnosis.weakest, data:
qai.dataset.stream("huggingface://secure_code_examples"))
```

Weaknesses vs. knowledge gaps — different problems, different fixes: a

weakness has partial exposure and performs badly; a gap has zero exposure to a whole topic.

```
model.knowledge_map() // → { security_code: "weak (0.42)",
  quantum_physics: "gap (no exposure)", cooking: "strong" }
model.finetune(data: quantum_data, mode: "adapter",
```

```
new_domain: true) // filling a genuine gap, not refining a
weak skill
```

Parameters vs. hyperparameters — clarified:

```
model.hyperparameters() // what the developer sets: layers,
heads, learning rate, epochs
model.parameters()      // what training LEARNED: actual
weight values — empty until training happens
```

44. Input/Output, Freeform Output & Technique Upgrading

Input / Output — simple, symmetric naming (renamed from the earlier "receiver" — same word pattern, obvious pair), same capability-checking discipline on both sides:

```
input Input {
  fn parse(raw) -> ModelInput
}
output Output {
  fn render(model_output) -> Artifact
}
qai.input("character.obj", as: "3d")
model.generate(prompt).output("image")
```

Freeform/experimental output mode — no renderer forced, model outputs its natural raw structure, useful for research/exploration, clearly marked as advanced/experimental rather than default:

```
model.generate(prompt, format: "freeform")
```

Custom bridges remain fully open — confirmed capable of arbitrary logic (e.g. an image → code translation bridge):

```
bridge ImageToCodeBridge extends Bridge {
  fn translate(image_output) -> code_input { /* fully
custom logic */ }
```

```
}
model_a.pipe_to(model_b, bridge: ImageToCodeBridge)
```

Upgrading a technique type without silently affecting existing models:

```
technique MoE_v2 extends MoE {
    fn new_capability() { ... }
}
model.upgrade_technique(MoE_v2)  // explicit opt-in per
model – nothing changes automatically
```

45. Parameter Reference — Technical Names & Simple Aliases

Every parameter has a plain-English alias alongside its technical name; both work interchangeably, and `param.help()` shows both together so beginners learn the friendly name first and pick up the standard term naturally.

```
model.set(experts: 32, active_experts: 4)  // simple alias
model.set(num_experts: 32, top_k: 4)       // technical name
– identical result
```

Core (every model): `input_size/output_size` (`input_dim/output_dim`), `inner_size` (`hidden_dim`), `depth` (`num_layers`), `attention_heads` (`num_heads`), `forget_rate` (`dropout`), `normalize` (`layer_norm`), `starting_weights` (`weight_init`), `precision` (`dtype`)

Transformer/LLM: `vocabulary_size` (`vocab_size`), `memory_span` (`max_seq_len/context_length`), `model_width` (`dim/d_model`), `position_scale` (`rope_theta`), `remember_past` (`kv_cache`), `share_vocabulary` (`tie_weights`)

MoE: `expert_count` (`num_experts`), `active_experts` (`top_k`), `always_on_expert` (`shared_expert`), `balance_strength` (`load_balancing_loss_coef`), `router_randomness` (`noisy_gating`)

CNN: `filter_size` (`kernel_size`), `step_size` (`stride`), `downsample_method` (`pooling`)

Diffusion: `noise_steps` (`num_timesteps`), `prompt_strength`
(`guidance_scale`)

Training/Optimizer: `learning_speed` (`learning_rate`), `weight_shrink`
(`weight_decay`), `warmup_period` (`warmup_steps`), `gradient_limit`
(`gradient_clipping`), `label_softness` (`label_smoothing`), `speed_mode`
(`mixed_precision`), `finetune_rank/finetune_strength` (`lora_r/lora_alpha`)

Regression/Classification: `category_count` (`num_classes`),
`output_style` (`output_activation`)

RL: `future_importance` (`gamma`), `exploration_rate` (`epsilon`),
`memory_size` (`replay_buffer_size`)

GAN: `creator_speed/critic_speed` (`generator_lr/discriminator_lr`),
`noise_size` (`latent_dim`)

RNN: `two_directions` (`bidirectional`), `memory_style` (`cell_type` —
LSTM/GRU/plain)

GNN: `hop_count` (`num_message_passing_layers`), `combine_method`
(`aggregation_type`)

Audio: `frequency_bands` (`n_mels`)

Tokenizer: `splitting_method` (`vocab_method` — `bpe/wordpiece`)

Ideas:

- `qai.glossary()` — prints the entire parameter reference table (simple name, technical name, meaning, typical range) as a built-in cheat sheet
- Difficulty tiers per parameter (`param.tier` : beginner/advanced) — a simplified build mode only surfaces beginner-tier params by default
- Smart defaults with reasoning shown in `.help()` ("dropout defaults to 0.1 — enough to reduce overfitting without slowing learning much")
- Typo-correction on parameter names ("did you mean `dropout` ?")
- Live parameter cheat cards in the dashboard with plain-English tooltips

46. Technique Types & Unique Methods (Full Reference)

Shared by every type: `.train()`, `.finetune()`, `.test()`, `.explain()`,
`.weights()`, `.loss()`, `.accuracy()`, `.parameters()`, `.visualize()`,
`.help()`

Unique methods per type:

Type	Unique methods
Transformer/LLM	<code>.generate()</code> , <code>.tokenize()</code> , <code>.attention_map()</code> , <code>.perplexity()</code> , <code>.kv_cache_stats()</code>
MoE	<code>.expert_routing()</code> , <code>.expert_utilization()</code> , <code>.balance_report()</code>
CNN	<code>.filter_visualize()</code> , <code>.feature_map(layer)</code> , <code>.receptive_field()</code>
Diffusion	<code>.set_seed()</code> , <code>.negative_prompt()</code> , <code>.img2img()</code> , <code>.inpaint(mask)</code> , <code>.controlnet_condition()</code> , <code>.upscale()</code> , <code>.style_transfer()</code> , <code>.interpolate()</code> , <code>.attention_slice()</code> , <code>.tile_mode()</code> , <code>.safety_filter()</code> , <code>.denoise_steps()</code> , <code>.guidance_adjust()</code>
Regression	<code>.formula()</code> , <code>.residual_plot()</code> , <code>.r_squared()</code>
Classification	<code>.confusion_matrix()</code> , <code>.roc_curve()</code> , <code>.class_probabilities()</code>
RNN/LSTM/GRU	<code>.hidden_state()</code> , <code>.unroll_view()</code> , <code>.sequence_memory_decay()</code>
GAN	<code>.discriminator_score()</code> , <code>.generator_score()</code> , <code>.mode_collapse_check()</code>

Type	Unique methods
RL	<code>.policy_visualize()</code> , <code>.reward_trace()</code> , <code>.exploration_report()</code>
GNN	<code>.graph_embedding()</code> , <code>.node_importance()</code> , <code>.message_passing_trace()</code>
K-Means/Clustering	<code>.cluster_centers()</code> , <code>.elbow_plot()</code> , <code>.silhouette_score()</code>
Autoencoder/VAE	<code>.encode()</code> , <code>.decode()</code> , <code>.reconstruction_error()</code> , <code>.latent_traverse()</code>
BERT-style/Encoder-only	<code>.mask_predict()</code> , <code>.sentence_embedding()</code> , <code>.similarity(a,b)</code>
Seq2Seq	<code>.beam_search(width)</code> , <code>.alignment_map()</code>
Decision Tree/Random Forest	<code>.tree_visualize()</code> , <code>.feature_importance()</code> , <code>.prune(depth)</code>
SVM	<code>.support_vectors()</code> , <code>.margin_width()</code> , <code>.kernel_transform()</code>
Naive Bayes	<code>.class_priors()</code> , <code>.likelihood_table()</code>
Hidden Markov Model	<code>.transition_matrix()</code> , <code>.viterbi_path()</code> , <code>.emission_probabilities()</code>
Genetic Algorithm	<code>.population_view()</code> , <code>.mutation_trace()</code> , <code>.fitness_landscape()</code>

Type	Unique methods
Self-Organizing Map	<code>.map_visualize()</code> , <code>.neighborhood_function()</code>
Hopfield Network	<code>.energy_landscape()</code> , <code>.stored_patterns()</code> , <code>.convergence_trace()</code>
Ensemble	<code>.member_contributions()</code> , <code>.voting_breakdown()</code> , <code>.diversity_score()</code>
Recommender/Collaborative Filtering	<code>.similar_items()</code> , <code>.user_embedding()</code> , <code>.cold_start_handle()</code>
Contrastive/CLIP-style	<code>.embed()</code> , <code>.similarity_matrix()</code> , <code>.zero_shot_classify()</code>
Energy-Based Models	<code>.energy_score()</code> , <code>.sample_via_langevin()</code>
Normalizing Flows	<code>.forward_flow()</code> , <code>.inverse_flow()</code> , <code>.log_likelihood()</code>
Speech Recognition (ASR)	<code>.transcribe()</code> , <code>.word_timestamps()</code> , <code>.confidence_per_word()</code>
Text-to-Speech (TTS)	<code>.synthesize()</code> , <code>.voice_clone()</code> , <code>.prosody_adjust()</code>
World Models (robotics/sim)	<code>.predict_next_state()</code> , <code>.rollout(steps)</code>

Framework-wide ideas:

- `model.available_methods()` — lists exactly which methods this specific model instance supports, no memorizing the table
- Technique family grouping — `qai.family("generative")` groups related types (Diffusion/GAN/VAE/Flows) so the technique advisor can compare within a family

- `.help()` with live example output — runs the method on the current model and shows a live result thumbnail, not just a text description
- Cross-type method borrowing — explicit opt-in only, e.g. a Regression model optionally borrowing a Classification-style diagnostic, never automatic

47. Output Control — Confirmed & Expanded

Max tokens/output limits were already part of the design (Section 15) and remain a core serving control:

```
model.serve(
    max_tokens: 200,
    stop_words: ["END", "\n\n"],
    required_words: ["def", "return"],
    banned_words: ["TODO", "pass"],
    temperature: 0.7
)
```

Extended with technique-appropriate equivalents beyond text:

- Diffusion → `.set_seed()`, `.guidance_adjust(scale)` (Section 46) act as the equivalent output-shaping controls for image/video generation
- Any generative type → `max_output_size` (bytes/resolution/duration cap) as the general non-text analog to `max_tokens`
- Per-role limits remain available for multi-role/self-play models (`asker.max_tokens`, `answerer.max_tokens`)

48. Federated Learning, Meta-Tooling & Cluster Training

```
qai.federated_train(model, devices: connected_devices,
    aggregation: "fed_avg") // train across devices, data never
centralizes
```

```

qai.describe("a model that classifies emails as spam or not,
fast, runs on CPU") // natural language → generated Quantum
spec, explainable

qai.train_cluster(model, nodes: 4, strategy: "data_parallel"
| "model_parallel") // multi-node orchestration

qai.label_session(unlabeled_data, ui: "local_web") //
built-in labeling tool, feeds active learning

model.deploy(strategy: "canary", initial_traffic: 5%,
ramp_to: 100%, rollback_on: "error_rate > 2%")

model.compliance_report(standard: "eu_ai_act") // audit
doc: provenance, bias, guardrails, robustness

model.personalize(local_data, privacy: "on_device_only")

model.export("model.wasm") // browser-native, client-side
inference

qai new lockfile // pins exact versions of every imported
technique/skill/dataset

```

- Notebook/playground mode — lighter, cell-by-cell interactive mode distinct from the full REPL, inline chart/image rendering per cell
- Voice/conversational model-building — speech input feeding into `qai.describe()`

49. Weight Averaging, Quantization-Aware Training & Data/Process Tools

```

let averaged = qai.average_weights([finetune_v1, finetune_v2,
finetune_v3]) // "model soup" – cheap alternative to
bridging

model.train(data, quantization_aware: true, target_precision:
"int4")

let synthetic = qai.generate_synthetic_data(from:

```

```

real_dataset, count: 10000, dedupe: true)

model.explainability_score()  // 0-100, standardized "how
much of a black box is this"

model.chaos_test(inject:
["corrupted_input", "missing_field", "extreme_values"], on:
"live_traffic_sample")

qai.simulate_users(model, count: 500, personas:
["confused_beginner", "adversarial", "power_user"])

qai.tutor(on: true)  // proactive live pair-programmer,
flags likely mistakes as you build

model.changelog(v1, v2)  // plain-English summary of what
changed between versions

model.export("model.gguf", license: "MIT", usage_terms: "non-
commercial")

```

- Accessibility-first dashboard — screen reader support, colorblind-safe palettes, full keyboard navigation as defaults
- Hosted sandbox/playground — try small models via a shareable link, no local install required
- Real-time collaborative architecture editing — multiple people editing the same model's layers simultaneously

50. Debate, Cascade Inference & Model Health Tools

```

qai.debate(model_a_answer, model_b_answer, judge: model_c)
// forces disagreement to surface reasoning weaknesses

model.train(data, time_budget: "2 hours",
checkpoint_on_timeout: true)

model.infer(input, cascade: [small_fast_model, medium_model,
large_model], escalate_if: "confidence < 0.7")

```

```

model.consistency_check(traits: ["formal tone", "no
emojis", "concise"]) // conversational drift detection

model.forensic_trace(bad_output) // full reverse search of
training examples that could explain a bad output

model.snapshot_as_tests(sample_inputs) // auto-generates a
regression test suite from current correct behavior

model.encyclopedia_score() // correct answers per dollar/watt
of compute

qai.import_from_pytorch("model.pt")
qai.import_from_tensorflow("model.h5")

model.schedule_retrain(every: "1 week", data:
qai.dataset.stream("latest_data"), auto_deploy_if:
"improves_accuracy")

model.set_refusal_threshold(confidence: 0.6) // refuses
rather than guesses below threshold

qai.registry.share(model, team: ["alice", "bob"], role:
"viewer" | "editor" | "owner")

```

- Model "mood" indicator — one-word training health summary (stable/struggling/diverging) combining loss trend, gradient stability, expert balance

51. Idle Training, Lifecycle & Cross-Model Tools

```

model.enable_idle_training(target: "weakest_ability",
max_cpu_percent: 20) // self-improves during idle compute

model.deprecate(reason: "superseded by v3", redirect_to:
model_v3, sunset_date: "2027-01-01")

qai.compare_concepts(model_a, model_b, concept: "fairness")
// checks semantic drift between models

model.autotrain(target: "accuracy", goal: 0.95, budget:

```

```

"$50", stop_at_budget: true)

model.serve(second_opinion: backup_model, trigger:
"confidence < 0.65")

model.generate(prompt, tone: "warm" | "neutral" | "assertive"
| "playful")

model.biography() // plain-language auto-written history of
the model's lineage

```

- Physical hardware companion mode — monitors temperature/throttling during long runs, warns before damage, not just a one-time spec check
- Community leaderboard opt-in for personal models — lighter, informal ranking among friends, separate from the formal benchmark integration (Section 25)
- Training-data consent flag — tracks whether user-generated content in a dataset is actually permitted for further fine-tuning, checked alongside the license check

52. Risk Scoring, Orchestration & Learning Tools

```

model.risk_score() // combines robustness, bias scan,
confidence calibration into one rating

model.predict(input).explain(level: "5th_grade" | "beginner"
| "expert")

model.shadow_deploy(candidate_model, compare_silently: true)
// parallel testing without affecting real traffic

qai.conductor([code_model, image_model,
chat_model]).route(request) // pure dispatch, distinct from
merge/bridge

model.hypothesis_test(new_example, show: "predicted_impact")
// "what if" without a real training run

qai.queue_experiments([config_a, config_b, config_c],
run_when: "idle")

```

```

qai.sketch(examples: [("hello","greeting"),
("bye","farewell")]) // infer setup from examples, not a
text description

model.lifetime_cost_estimate(expected_requests_per_month:
100000, over: "1 year")

qai.learning_mode(student: true) // achievement milestones
tied to real understanding

qai.doctor() // Quantum's own
installation/compiler/hardware-driver health check

model.pitch() // one-paragraph plain-language summary for
sharing outside the framework

```

- Multi-tenant isolation — strict memory/compute isolation between users' models if ever hosting multiple people's work on shared infrastructure

53. Expanded Memory & Context System

Full memory parameter table, with simple aliases (extends Section 45):

Technical	Simple alias	Meaning
max_seq_len / context_length	memory_span	total text the model can consider
sliding_window / window_size	focus_width	how much it actually looks at per token, inside that span
rope_theta	position_scale	how it tracks word order over distance
rope_scaling	stretch_factor	extends trained context length beyond training

Technical	Simple alias	Meaning
<code>use_kv_cache</code>	<code>remember_past</code>	reuses prior computations during generation
<code>kv_cache_dtype</code>	<code>cache_precision</code>	memory/speed tradeoff for the cache
<code>num_kv_heads</code>	<code>shared_memory_heads</code>	fewer than query heads = less memory (GQA)
<code>chunk_size</code> / <code>chunk_overlap</code>	<code>retrieval_bite_size</code> / <code>retrieval_overlap</code>	how RAG splits documents
<code>top_k_retrieval</code>	<code>docs_recalled</code>	how many external documents get pulled in
<code>ssm_state_size</code> (Mamba)	<code>flow_memory_size</code>	state-space model's compressed running memory
<code>conv_kernel</code> (Mamba)	<code>local_window</code>	small local pattern size
<code>expand_factor</code> (Mamba)	<code>width_multiplier</code>	internal width expansion
<code>sparse_attention</code>	<code>selective_focus</code>	attends to only some tokens by pattern
<code>sparsity_pattern</code>	<code>focus_pattern</code>	strided/fixed/learned selection method
<code>mixture_of_depths</code>	<code>skip_layers</code>	dynamically skips layers per input
<code>attention_dropout</code> / <code>hidden_dropout</code>	<code>focus_forget_rate</code> / <code>layer_forget_rate</code>	dropout on attention vs. general layers

Technical	Simple alias	Meaning
<code>stochastic_depth</code>	<code>layer_skip_rate</code>	randomly skips whole layers in training
<code>mixup_alpha</code>	<code>blend_strength</code>	blends two training examples together
<code>cutmix</code>	<code>patch_swap</code>	vision-specific patch swapping
<code>gradient_checkpointing</code>	<code>memory_saver_mode</code>	trades compute time for lower training memory
<code>ema_decay</code>	<code>smoothing_factor</code>	smoothed running average of weights
<code>z_loss_coef</code>	<code>logit_stabilizer</code>	prevents unstable output score growth
<code>beta_kl</code>	<code>alignment_pull</code>	how strongly kept close to original behavior during RLHF
<code>reward_clip</code>	<code>reward_limit</code>	caps extreme RLHF reward values
<code>ppo_clip_ratio</code>	<code>update_limit</code>	caps how much one RLHF update can shift behavior
<code>preference_loss</code>	<code>preference_method</code>	DPO/ORPO/KTO alignment approach
<code>reference_model</code>	<code>comparison_baseline</code>	frozen copy for measuring alignment drift
<code>retriever_type</code>	<code>search_style</code>	dense/sparse/hybrid retrieval

Technical	Simple alias	Meaning
<code>embedding_dim</code>	<code>vector_size</code>	retrieval embedding vector size
<code>reranker_model</code>	<code>result_sorter</code>	re-orders retrieved documents by relevance
<code>vector_db</code>	<code>memory_store</code>	database holding retrieval vectors
<code>quantization</code>	<code>compression_type</code>	int8/int4/GPTQ/AWQ weight shrinking method
<code>bits</code>	<code>precision_bits</code>	detail per weight (4 or 8)
<code>group_size</code>	<code>compression_block</code>	weights compressed together per group
<code>pruning_sparsity</code>	<code>trim_ratio</code>	fraction of weights removed
<code>lora_target_modules</code>	<code>finetune_targets</code>	which layers a fine-tune may touch

Allocating memory for a specific model – three levels of control:

```
// 1. Preset – simplest, framework configures everything underneath
model.set_memory(preset: "long_chat") // → sliding window + RoPE scaling + int8 KV cache + RAG, bundled

// 2. Explicit – allocate each feature individually
model.memory.configure(
    span: 32000, focus_width: 4096, position_scale: 500000,
    remember_past: true, cache_precision: "int8"
)
model.memory.enable("retrieval", docs_recalled: 5,
    retrieval_bite_size: 512,
    source: "huggingface://my_knowledge_base")
```

```
// 3. Per-tier — adjust just one piece of an existing model
existing_model.memory.enable("retrieval", docs_recalled: 3)
existing_model.memory.disable("sliding_window")
existing_model.memory.status() // → current allocation
snapshot

// 4. Advisor — framework recommends, developer approves
explicitly (never silent)
let suggestion = qai.suggest_memory_strategy(target_context:
"32k", hardware: qai.detect_hardware())
model.memory.apply(suggestion)
```

Memory visualization suite:

- Sliding window visualizer — animated highlighted box moving across tokens, showing exactly what's "in focus" per generated token
- Context-window fill meter — visible bar showing how full the window is; tokens fall off the left edge visibly once full
- KV cache growth graph — live memory usage climbing during generation
- Attention distance heatmap — how far back each generated token actually attended
- RoPE position wheel — circular/spiral visual of positional encoding rotating through embedding space, and how `stretch_factor` extends it
- Memory-tier flow diagram — live trace of which tier answered a given query (parametric / retrieval / context window), color-coded
- RAG retrieval highlight — shows exactly which document chunk got pulled in and how it influenced the answer
- Memory decay/fade effect — tokens near the window edge rendered dimmed, tokens outside grayed out entirely

More memory ideas:

- Per-conversation memory checkpointing — pause/resume a specific conversation's working memory independently of model weights
- Memory compression scheduling — auto-summarize older context instead of simply dropping it when the window fills

- Cross-session memory merge — combine retrieval learnings across separate sessions into one updated store
- Memory access permissions — control read/write access to a shared retrieval store across multiple users/agents
- Adaptive window sizing — `focus_width` auto-adjusts to content complexity instead of staying fixed
- Forgetting-curve customization — important facts persist longer in working memory, routine details fade faster, instead of uniform aging
- Memory encryption at rest for the retrieval store, given it may hold sensitive personal data
- Shared memory pool between multiple models — e.g. self-play roles (Section 14) optionally sharing one retrieval pool instead of isolated stores
- `model.memory.audit()` — log of exactly what was retrieved, when, and why, for any past response

54. Behavior Contracts, Drift Monitoring & Production Lifecycle Tools

```
model.set_contract(min_accuracy: 0.90, max_latency_ms: 200,
max_refusal_rate: 0.05)
model.contract_status()    // "meeting" | "breached: latency
340ms > 200ms"

model.monitor_drift(baseline: training_data_stats, alert_if:
"distribution_shift > threshold")

model.undo(steps: 50)    // step-level rollback, finer than
checkpoints

model.silent_failure_check()    // flags confidently-wrong
patterns, distinct from calibration

model.known_limitations()    // model self-reports its own
weak areas in plain language, from real weakness/gap data

model.pareto_view(objectives: ["accuracy","speed","size"])
// visualize tradeoffs instead of one number
```

```

model.at_checkpoint("epoch_5").predict(input)  // query a
past version's behavior without rolling back

qai.workspace.share_component(rigging_skill, from:
"project_a", to: "project_b")

qai.registry.find_similar(model)  // finds
architecturally/weight-similar models already in your
registry

model.infer_batch(large_input_list, parallel: true)

```

- Guided permissions wizard for agentic models — step-by-step flow generating safe `egress / sandbox` config instead of hand-writing it
- Localization of the language/dashboard itself — error messages, `.help()`, and labels translatable

55. Marketplace, Statistical Rigor & Stakeholder Tools

```

qai.registry.publish(model, price: "$5/1000 requests",
license: "commercial")

model.compare_significance(v1, v2, metric: "accuracy",
confidence_level: 0.95)
// → "v2 is significantly better (p=0.02), not just noise"

model.time_capsule("2026-full-snapshot")  // full
environment freeze:
data+code+weights+hyperparams+dependencies

model.interview(questions: domain_specific_test_set)  //
structured pass/fail screening before trusting a model

model.monitor_user_sentiment(flag_if:
"frustration_pattern_detected")

qai.dataset.mark_golden("core_eval_set", immutable: true)
// protected reference eval set, never touched by training

```

```

model.set_mistake_weights({"missed_critical_bug": 10,
"minor_typo": 1})
model.train(target: "weighted_accuracy")    // not all wrong
answers are equally bad

model.generate_onboarding_doc()

qai.check_parity(model, environments:
["dev", "staging", "prod"])

qai.dashboard(mode: "stakeholder")    // business-outcome
view, hides loss curves/gradients

qai.merge(model_a, model_b, personality_source: "model_a")
// whose tone/style dominates after merging

model.train(data, on_failure: "suggest_alternative")
// → "Regression isn't converging – looks non-linear. Try
'gradient_boosting'?" (suggestion only, never automatic)

```

56. Type/Technique Authoring System

`type` and `technique` are interchangeable keywords – same concept, whichever reads naturally.

Every Type decomposes into swappable mechanics, not one fixed formula:

```

type Regression {
  structure { layers: [linear] }           // what
  makes it "this" Type specifically

  uses optimizer: GradientDescent          //
  swappable: Adam, SGD, Lion...

  uses objective: MSE                      //
  swappable: MAE, Huber, CrossEntropy...

  uses training_loop: RetrainUntil(loss < 0.1) //
  swappable: Once, EpochBased(n), TimeBoxed(duration)
}

type MyRegression extends Regression {
  uses optimizer: Adam
}

```

```

    uses training_loop: RetrainUntil(loss < 0.05,
max_attempts: 10)
}

```

- `Optimizer`, `Objective`, and `TrainingLoop` are their own open interfaces/registries, shared across every Type — a CNN and a transformer can both use `GradientDescent` + `RetrainUntil(...)` while keeping their own unique `structure`
- Mechanics compatibility check — e.g. a non-differentiable Type can't use a gradient-based optimizer; errors clearly rather than failing silently
- "Mixing board" dashboard view — swap optimizer/objective/loop live and watch the effect, rather than editing code
- Named loop presets — `TrainingLoop.retrain_until_convergence()`, `.retrain_n_times(5)`, `.train_once()`

Full Type definition anatomy:

```

type Regression {
  meta { name: "Regression", family: "classical_ml",
description: "...", version: "1.0" }

  capabilities { input: ["tabular","numeric"], output:
["numeric_prediction"], supports_weights: true }

  params {
    learning_rate: f64 { alias: "learning_speed",
default: 0.01, range: [0.0001, 1.0] }
  }

  fn forward(input) -> output { ... }
  fn objective(output, target) -> Signal { ... }
  fn update(signal) { ... }

  on_init { ... }
  on_epoch_end { ... }
  on_train_end { ... }

  fn formula() -> Equation { ... }           // unique method

  validate {
    case("simple linear data") {
train_on(known_linear_dataset); assert(accuracy > 0.95) }

```

```
}  
}
```

Each block auto-populates a different system: `meta` → registry/family grouping; `capabilities` → Input/Output matching (Section 44); `params` → `.help()`, `qai.glossary()`, dashboard cheat cards (Section 45); lifecycle hooks → dashboard/autosave/anomaly alerts wire in automatically; `validate` → required before a Type is marked verified.

Lifecycle states: `draft` → `validated` (passes `validate{}`) → `certified` (benchmarked, Section 25) → `published` (registry) → `deprecated`. `type.status()` shows current state; unverified custom types are flagged visibly wherever they're used.

Composition:

```
type MyRegression extends Regression { ... }  
type Hybrid combine(CNN, Transformer) { ... }  
type Rigging requires(MeshGeneration) { ... }
```

Additional Type families worth adding as real built-ins:

- `SpeculativeDecoding { draft_model, verifier }` — fast draft model proposes tokens, larger model verifies in one pass
- Meta-learning / few-shot (MAML-style) — learns *how to learn quickly* from very few examples
- Causal inference — cause-and-effect estimation, distinct from correlation-based prediction
- Anomaly detection — first-class family for fraud/defect/outlier detection
- Learning-to-rank — search/recommendation-specific, optimizes order not a single prediction
- Time-series forecasting — seasonality/trend-aware tools, distinct from generic regression

57. Formula Reference Library

Every core Type's defining formula, rendered and explorable:

Type	Core formula
Regression	$y = mx + b$
Logistic Regression	$p = 1/(1+e^{-(w \cdot x+b)})$
Decision Tree	$Gini = 1 - \sum p(i)^2$
SVM	maximize margin = $2/\ w\ $
K-Nearest Neighbors	distance = $\sqrt{\sum (x_i - y_i)^2}$
Naive Bayes	$P(class data) = P(data class) \cdot P(class)/P(data)$
K-Means	minimize $\sum \ x - centroid\ ^2$
Perceptron	output = step($w \cdot x + b$)
CNN	$(f * g)(t) = \sum f(\tau)g(t - \tau)$
RNN	$h_t = \tanh(W \cdot h_{t-1} + U \cdot x_t)$
Transformer (Attention)	Attention(Q, K, V) = $\text{softmax}(QK^T/\sqrt{d}) \cdot V$
MoE (Routing)	output = $\sum g_i(x) \cdot E_i(x)$
GAN	$\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$
VAE/Autoencoder	Loss = reconstruction_error + $KL(q(z x) \ p(z))$
Diffusion	$x_{t-1} = x_t - \epsilon_{\theta}(x_t, t)$
RL (Bellman)	$Q(s, a) = r + \gamma \cdot \max_{a'} Q(s', a')$
GNN (message passing)	$h_v = \text{update}(h_v, \sum \text{message}(h_u))$
Markov Chain	$P(X_{n+1} X_n, \dots, X_1) = P(X_{n+1} X_n)$
Hopfield Network	$E = -\frac{1}{2} \sum \sum w_{ij} \cdot s_i \cdot s_j$

Type	Core formula
PCA	finds eigenvectors of the covariance matrix

```
qai.formula_library()           // browse every
built-in formula, searchable
model.formula(mode: "math" | "code" | "plain_english")
```

- Formula comparison view — two Types' formulas side by side
- Live formula substitution — after training, shows the equation with actual learned numbers plugged in
- Formula-to-code trace — click a symbol, jump to where it's defined in the model's parameters
- Formula history across training — see the filled-in formula change from epoch 1 to epoch 100

58. Advanced Visualization — Formula Editing, Probability & Confidence

```
let editor = model.formula().edit()
editor.set(m: 2.5, b: 1.0)      // manually adjust the fitted
equation
editor.preview_loss()           // see instantly how "wrong"
the manual guess is
editor.reset_to_learned()
```

```
model.generate_stream("the cat is very",
  on_token: fn(candidates) { candidates.show() } // live
bar chart: fat 62%, cute 21%, fast 9%...
)
```

- Per-layer confidence trace — how "sure" the model is at each internal layer, not just the final output
- Probability calibration curve — checks if "80% confident" predictions are actually right ~80% of the time (visual form of Section 26)

- Probability distribution shape view — sharp (95/5) vs. flat (30/30/25/15) distributions visualized, distinguishing confidence from genuine uncertainty

59. Universal Manipulation System

One consistent chainable pattern across every modality, all capability-checked and fork-able like everything else:

```
manipulation Manipulation {
  fn required_capability() -> String
  fn apply(input) -> output
}
```

Text: `.format()`, `.summarize()`, `.translate()`,
`.adjust_reading_level()`, `.redact()`, `.highlight()`, `.diff()`,
`.word_count_limit()`

Audio: `.trim()`, `.normalize_volume()`, `.pitch_shift()`, `.speed()`,
`.reduce_noise()`, `.fade()`, `.isolate()`, `.add_reverb()`

3D: `.scale()`, `.rotate()`, `.smooth_mesh()`, `.decimate()`,
`.unwrap_uv()`, `.rig()`, `.generate_lods()`, `.add_collision_mesh()`

Documents: `.add_table_of_contents()`, `.paginate()`, `.watermark()`,
`.extract_tables()`, `.compress()`

Tabular: `.filter()`, `.pivot()`, `.normalize_columns()`,
`.one_hot_encode()`, `.handle_missing()`, `.join()`

Code: `.format()`, `.add_comments()`, `.add_type_hints()`,
`.extract_function()`, `.minify()`

Image/Video (recap): `.crop()`, `.resize()`, `.rotate()`, `.focus_mask()`,
`.zoom()`, `.sync_lips()`, `.stabilize()`, `.trim()`

Supporting ideas:

- Manipulation history/undo stack — reversible one step at a time
- Non-destructive editing — original generation always recoverable via `.reset_to_original()`
- Named presets/macros — save a whole chain as one reusable tool (`qai.save_preset(...)`, `.apply_preset(...)`)
- Batch manipulation — apply one chain across many outputs at once

- Before/after diff preview
- Manipulation cost estimator for compute-heavy operations
- Cross-modal manipulation search —
`model.history.find(manipulation: "resize")`
- Manipulation marketplace, same registry pattern as techniques/skills

60. Unified Input/Output System (Settled Version)

No separate "transform" concept — everything routes through the same `Input / Output` pair, whether at training-data-prep time or live inference time. Invisible and zero-config by default; only surfaced when a developer actively wants control.

```
// Zero config — the default, for everyone who doesn't need
// to touch endpoints
model.train(data)
model.generate(prompt)

// Training-time shaping — model genuinely LEARNS the
// pattern, not cosmetic
data.on_input(fn(example) { example.image =
example.image.crop(region).resize(256,256); return example })
data.on_output(fn(example) { example.output =
example.output.upper(); return example })
model.train(data)

// Inference-time — same two endpoint names
model.input("photo.jpg", as: "image")
model.generate(prompt).output("json" | "plain_text" |
"markdown")
```

Optional, dial-able validation at both endpoints — never forced:

```
model.input.validate(level: "none" | "basic" | "strict")
model.output.validate(level: "none" | "basic" | "strict")
```

`basic` is the sane default everywhere; `strict` (full Section 33/35 security scanning) is opt-in for production/security-sensitive cases, since an

aggressive default could silently reject legitimate unusual input and frustrate beginners.

.chat() — instant auto-generated UI, no HTML/JS written by hand:

```
model.chat() // opens a real browser chat window, or an
upload-and-predict form for a classifier, etc.
```

This is the simplest case of `qai.dashboard()` (Section 19) — the framework already knows the model's input/output shape and generates the right interface automatically.

Multi-layer/mixed-type models — still one door in, one door out from the outside:

```
let model = qai.build()
model.add_layer("rnn_part", type: "rnn")
model.add_layer("transformer_part", type: "transformer")
model.add_layer("regression_head", type: "regression")
model.connect("rnn_part" -> "transformer_part" ->
"regression_head")

model.input(raw_data) // ONE entry point for the whole
model
model.generate(prompt) // ONE exit point — internal
layers stay private
```

Advanced, opt-in internal taps remain available using the exact same vocabulary, applied at a deeper level:

```
model.layer("transformer_part").input.watch()
model.layer("rnn_part").output.watch()
```

Supporting checks:

- `model.check_endpoints()` — verifies training-time input/output shaping matches what's expected at inference time, catching train/serve mismatches
 - `model.endpoint_log()` — one place listing every input/output manipulation ever applied, across both training prep and live serving
-

61. Error Handling, Approval Gates, Privacy & Platform Reach

Error/recovery handling for AI operations — currently missing a real language-level recovery path:

```
try {
  model.train(data)
} catch (LossDiverged) {
  model.rollback_to_last_checkpoint()
  model.train(data, learning_rate: lower)
} catch (OutOfMemory) {
  model.train(data, batch_size: smaller)
}
```

Human-in-the-loop approval gates — pauses for actual human sign-off, distinct from automatic guardrails:

```
model.require_approval(action_type: "send_email" |
  "delete_file" | "spend_money")
```

Privacy-preserving/encrypted inference — the inference-time counterpart to training-time differential privacy (Section 33):

```
model.infer(input, mode: "encrypted")  // computation
without ever decrypting the input
```

Cross-language bindings — lets a trained model be called from outside Quantum entirely:

```
model.export_bindings(target: "python" | "javascript" |
  "c_ffi")
```

Crisis/safety detection for conversational models — its own category, distinct from topic-blocking guardrails; detects distress in the *user's* input and routes to a careful, resource-surfacing response:

```
model.serve(safety_layer: "crisis_detection")
```

Full package manager CLI, on top of the Section 42 lockfile:

```
qai search "rigging"
qai install rigging_skill@2.1
qai update
qai uninstall rigging_skill
```

Dynamic multi-model routing (learns over time) – distinct from the static conductor (Section 52):

```
qai.dynamic_router([model_a, model_b, model_c], strategy:
"bandit")
```

Data augmentation toolkit, detailing the Section 12 mention:

```
data.augment(["rotate","flip","add_noise","back_translate","s
ynonym_swap"])
```

On-device inference energy tracking, distinct from training energy cost (Section 26):

```
model.energy_per_inference()
```

Priority note: error handling, approval gates, and crisis detection are the ones worth treating as must-have, not optional polish – they're the class of gap that causes real damage in production if left unaddressed.

62. Legal Compliance, Developer Tooling & Platform Depth

Right-to-be-forgotten as an auditable compliance workflow, not just the technical unlearning primitive (Section 33):

```
model.forget_user(user_id)
model.forget_confirmation(user_id)  // proof/audit record
the data's influence was actually removed
```

IDE integration — real tooling around the Section 39 language server mention:

```
qai.vscode_extension() // syntax highlighting, inline
                        dashboard previews, live training status in-editor
```

Conditional training breakpoints — waits silently for a condition, sharper than epoch-by-epoch replay (Section 6):

```
model.breakpoint(if: "loss > previous_loss * 1.5")
model.breakpoint(if: "expert_4_utilization < 0.01")
```

General-purpose standard library — a flag, not a feature: everything designed so far is AI-specific, but Quantum is a full language and needs ordinary file I/O, string handling, networking, date/time, and non-AI error handling to actually replace Python day-to-day, not just for AI tasks.

Collaborative editing conflict resolution — what happens when two people edit the same layer at once (extends Section 49):

```
model.resolve_conflict(edit_a, edit_b, strategy: "manual" |
"last_write_wins" | "merge_compatible")
```

Cross-compilation to embedded/microcontroller targets — a distinct, harder tier beyond mobile/edge (Section 12), relevant if physical-AI/IoT ever becomes a direction.

Project Status Note

As of Section 62, this document has moved from "core design" into its long tail — later additions (breakpoints, embedded compilation, legal workflows) are real but progressively more specialized and less frequently needed than the foundational sections (1–20). Treat Sections 1–47ish as the load-bearing core; everything past that is a genuine but optional backlog. Recommended next step: pick one small vertical slice (one Type, train → inspect → finetune → serve, one dashboard) and get it fully working end-to-end before pulling in more of this backlog — the document is a reference to build toward, not a checklist to complete before starting.

63. Optimization, Model Families & Capability Extraction

Syntax correction: built-in Types never need importing –

`qai.build(technique: "regression")` already knows what "regression" means. `import` is only for someone else's published custom technique:

```
let model = qai.build(technique: "regression") // no import
needed for built-ins
import "community/my_custom_technique" // only for
third-party custom techniques
let model = qai.build(technique: MyCustomTechnique)
```

Pruning — automatic by default; the developer sets the dial and scope, not individual weights:

```
model.prune(target: 0.3)
// "remove 30%" – framework decides which weights
model.prune(target: 0.3, scope: "attention_layers")
// coarse control: WHERE
model.prune(target: 0.3, method: "magnitude" |
"gradient_based" | "movement") // WHICH importance metric
model.prune_preview(target: 0.3)
// shows affected regions before committing
```

Real practice: an automatic importance score picks which weights go; hand-picking individual weights isn't realistic at scale.

Order-aware optimization pipeline (replaces the earlier flat option list):

```
let plan = qai.suggest_optimization(model, target: "runs on
4GB phone, keep accuracy above 90%")
// → recommends order: prune 25% → recovery finetune →
quantize int8 → fuse layers, with reasoning shown

model.optimize(plan, keep_uncompressed_backup: true)
model.optimize_report() // size/speed/accuracy tradeoff,
before vs after

// Manual, step by step:
```

```

model.prune(0.25)
model.finetune(data, epochs: 1)  // recovery finetune,
standard practice after lossy compression
model.quantize("int8")
model.fuse_layers()

```

Model families — one base model, multiple deployable sizes (the Gemini/Gemini Flash pattern):

```

let family = qai.create_family(base_model,
  variants: [
    { name: "large", size: "100%" },
    { name: "flash", size: "20%", method: "distillation" },
  ],
  { name: "nano", size: "5%", method: "distillation" }
)
family.large / family.flash / family.nano

```

Capability extraction — surgical, not distillation: identifies which weight regions actually drive a specific capability (via concept-decomposition, Section 4) and discards everything else, rather than retraining a smaller student.

```

let coding_only = model.extract_capability("coding")

```

Should warn if the target capability is entangled with others (shares weights, can't be cleanly separated).

64. Model Decomposition, Weight Editing & Skill Mechanics (Clarified)

.edit() — a real, concrete spec for direct manipulation of one decomposed section:

```

let section = model.decompose(by: "layer" | "magnitude" |
  "concept")["layer3"]
let editor = section.edit()

```

```

editor.set_weight(index: [0,5], value: 0.82)
editor.zero_out(indices: [12, 45, 90])
editor.scale(factor: 0.5)
editor.replace_with(other_section)
editor.commit()

```

Sections overview — see every decomposed section at once:

```

model.sections_overview()
// → { "layer1": {role: "input processing", weight_health:
"stable"},
//     "layer3": {role: "reasoning", weight_health: "some
dead weights"},
//     "output_layer": {role: "decision", skills_attached:
["rigging"]} }

```

`sections["x"].diff_from_original()` shows exactly what changed in one edited piece.

Skill mechanics — clarified precisely. A skill does NOT retrain the base model. The base model's weights are read from, never written to. A skill is a tiny separate calculation added at one declared point in the forward pass:

```

skill_output = base_representation + (skill_matrix_A ×
skill_matrix_B × base_representation)
final_output = base_model.continue_from(skill_output)

```

- Training a skill = only its own small A/B matrices update via backprop
- Attaching = adding that small calculation into the forward pass at its declared `insertion_point`
- Detaching = removing it — base model reverts exactly to its pre-attachment state

Full worked example:

```

let model = qai.build(technique: "diffusion", output_type:
"3d")
model.train(mesh_dataset)    // good at generating characters,
can't rig them

let rigging_skill = qai.skill.train(
  base: model, data:

```

```

qai.dataset.stream("huggingface://rigged_characters"),
    insertion_point: "output_layer", epochs: 5
) // base frozen throughout – only the skill's own adapter
weights train

model.attach_skill(rigging_skill)
model.generate("a walking character").rig()

```

qai.suggest_insertion_point(base_model, skill_goal: "rigging")
can recommend where a new skill should tap in.

65. Get/Set Method Discipline (Every Type)

A consistent rule across every Type: measured results are read-only;
settings/dials are adjustable, before or during training.

```

model.loss() // GET – read-only, a
measured outcome
model.accuracy() // GET – read-only

model.set_learning_rate(0.0005) // SET – adjustable
live, mid-training
model.set_target_epochs(50) // SET – adjustable
before or during training
model.pause_at_epoch(20) // SET – a live control
model.setting_history("learning_rate") // log of every mid-
training adjustment made, with timestamps

```

This mirrors the parameters-vs-hyperparameters distinction (Section 43)
applied consistently to every method, not just those two.

66. Live Streams, Learning Paradigms & Current Generative Techniques

Live raw input – an ordinary Input stream, no learning implied:

```

let camera_feed = qai.input.connect_stream(source: "camera",
type: "video")

```

```

model.process(camera_feed) // pure
inference on a continuous feed
model.online_learn(feed: camera_feed, memory_buffer: true)
// OPT-IN separate layer if weights should also keep updating

```

Online/incremental learning updates model parameters continuously from a live stream rather than a fixed dataset; real systems must guard against catastrophic forgetting (losing old knowledge while adapting) and concept drift (the real-world patterns themselves shifting) — `memory_buffer: true` periodically re-tests against old data to guard against this.

Type vs. Technique vs. Learning Paradigm — three related but separate concepts:

```

let model = qai.build(type: "regression") //
ARCHITECTURE — Regression/CNN/RNN/Transformer/MoE
model.learning_technique() // → "supervised", a sensible
default, always overridable

let model = qai.build(type: "transformer",
learning_technique: "self_supervised")

```

Each Type declares which learning paradigm(s) it's compatible with; mismatches produce a clear, corrective error, e.g.:

```

let model = qai.build(type: "transformer")
model.train(image_dataset)
// → Error: "Transformer's default input capability is
'text'. This dataset is images.
// Did you mean type: 'diffusion' or type:
'vision_transformer'?"

```

Full learning-technique list: Supervised, Unsupervised, Self-Supervised, Semi-Supervised, Reinforcement, Online/Incremental, Transfer Learning, Few-Shot/Meta-Learning, Contrastive Learning, Imitation Learning, Curriculum Learning (as a full paradigm, not just a training-loop detail). `qai.learning_techniques_library()` browses all of them with which Types they typically pair with.

Current (2026) generative techniques beyond diffusion, as real distinct Types:

```

type FlowMatching extends Technique { }      // learns a
direct noise-to-data vector field; now the default paradigm
for image generation, more efficient than diffusion
type RectifiedFlow extends FlowMatching { }  // near-
straight noise-to-data paths, fewer inference steps
type ConsistencyModel extends Technique { }  // maps noise
directly to data in far fewer steps; trainable from scratch
or distilled from diffusion

```

Video generation is generally image generation extended with a temporal dimension, often using a 3D VAE for combined spatial-temporal compression to keep frames consistent over time.

```

qai.family("image_video_generation")
// → Diffusion, FlowMatching, RectifiedFlow,
ConsistencyModel, GAN, Autoregressive_pixel_space, VAE_based

```

67. Bidirectional Model Communication & Multi-Model Networks

Bidirectional exchange ("debate"/"collaborate") — distinct from one-way

.pipe_to() :

```

let channel = qai.connect_bidirectional(model_a, model_b,
mode: "debate" | "collaborate" | "negotiate")
let result = channel.run(topic: "...", max_rounds: 5, until:
fn(history) { history.last_two_agree() })

```

Explicit, developer-controlled multi-model wiring — no assumed shape:

```

let network = qai.network()
network.add(model_a, model_b, model_c)
network.route(model_a -> model_b)
network.route(model_b -> model_c)
network.route(model_c -> model_a)    // full loops allowed
network.rewire(model_b -> model_a)   // change routing
later, anytime

```

Output ownership — an explicit choice per pipeline, never assumed:

```
qai.pipeline([model_a, model_b,
model_c]).designate_output(model_c)           // one model's
output is THE answer
qai.pipeline([model_a, model_b,
model_c]).combine_outputs(strategy: "merge")  // all
contribute, merged
qai.pipeline([model_a, model_b,
model_c]).combine_outputs(strategy: "bundle") // kept
separate (Bundle, Section 38)
qai.pipeline([model_a, model_b,
model_c]).combine_outputs(strategy: "vote")   // majority-
agreement answer
```

Conditional routing is just ordinary code — no special DSL needed, since models are regular callable objects:

```
let decision = director_model.decide(topic)
if decision.needs_image {
  let img = image_model.generate(decision.image_prompt)
  output.attach(img)
}
```

LearningTechnique as its own formal interface, parallel to `Type` (Section 56), with the same editable/read-only discipline as Section 65:

```
learning_technique ReinforcementLearning {
  params { reward: f64 {editable:true}, penalty: f64
{editable:true}, gamma: f64
{alias:"future_importance",editable:true}, epsilon: f64
{alias:"exploration_rate",editable:true} }
  readonly { cumulative_reward: f64, episode_count: int }
}
learning_technique TransferLearning {
  params { source_model: Model, frozen_layers: [string],
transfer_amount: f64
{alias:"carry_over_strength",editable:true} }
}
model.learning.reward = 10           // editable
model.learning.cumulative_reward()   // read-only, measured
```

General feedback-loop primitive — generalizes long-form video continuation (Section 22) to any "feed output back into itself" pattern (long story, extending image sequence, etc.):

```
qai.feedback_loop(
  model: video_model,
  carry: "last_frames",
  until: fn(state) { state.total_duration >=
target_duration },
  on_iteration: fn(chunk, state) { state.append(chunk) }
)
```

Consistency as a general, explicit model property — not buried inside one technique:

```
model.consistency_reference
model.set_consistency_strength(0.8)
```

68. Editing Core Model Logic ("Brain Logic")

Changing a model's actual internal logic — not just its parameters or hyperparameters — is done through `override`, already part of the Type system (Section 56), made concrete:

```
type VideoDiffusion {
  fn forward(current_frame) -> next_frame { /* default:
predicts forward */ }
}
type ReverseFramePredictor extends VideoDiffusion {
  override fn forward(current_frame) -> previous_frame { /*
genuinely different logic, not just different weights */ }
}
```

`override fn forward(...)` changes what KIND of thinking happens; training then finds good weights for that new logic — logic and learned weights are two separate things, and this is how you touch the first one.

Exporting custom-logic models — must bundle logic + weights, not weights alone:

```

model.export("my_model.qmodel")           // bundles custom
logic (Type definition/code) AND weights together
model.export_weights_only("my_model.gguf") // ONLY valid for
unmodified built-in Types, where the logic is already known
on the receiving end

```

Automated logic-variant search — tests structural variations of core logic automatically, not just hyperparameters. *Corrected in Section 70: takes real, user-authored candidate Types, never a hardcoded preset list of mutations.*

```

type MyIdeaA extends VideoDiffusion { override fn
forward(frame) -> next_frame { /* predicts backward, author's
own idea */ } }
type MyIdeaB extends VideoDiffusion { override fn
forward(frame) -> next_frame { /* a different author idea */
} }

let variants = qai.search_logic_variants(
  base: VideoDiffusion,
  candidates: [MyIdeaA, MyIdeaB],
  evaluate_on: video_dataset,
  metric: "consistency_score"
)
variants.best()

```

Logic sandbox — cheap preview before committing to a full retrain:

```

let sandbox = model.logic_sandbox()
sandbox.try_override("forward", new_logic_fn)
sandbox.preview(sample_input) // approximate result using
existing weights, no full retrain
sandbox.commit()              // now real, triggers
proper retraining

```

69. Logic Variant Management & Safety

```

model.logic_version()           // →
"ReverseFramePredictor v2, forked from VideoDiffusion"
output.logic_trace()           // which

```

exact logic variant produced this specific result

```
qai.serve(model_variants: [VariantA, VariantB],
traffic_split: [80, 20], compare_on: "consistency_score")

VariantB.logic_diff(VariantA)           // →
"forward() rewritten: now predicts t-1 instead of t+1"

network.set_cycle_limit(max_passes: 10)  // prevents
an A→B→C→A wiring from running forever

model.logic_sandbox().safety_check()     // flags if
new logic breaks basic type/shape guarantees before commit

channel.export_transcript("debate_log.md") // save a
full bidirectional exchange as a readable artifact

qai.network_topology_view()              // live
diagram of which models are wired to which

qai.import_logic("community/reverse_frame_prediction") //
shareable logic variants, same registry pattern as
skills/techniques

// auto-stop instead of hanging if a feedback/debate loop
never converges
qai.feedback_loop(model: x, until: condition, max_iterations:
100, on_timeout: "report_and_stop")

model.consistency_score(output_sequence) //
objective, standardized measure of consistency across long
outputs
```

70. Open Logic Authoring — No Hardcoded Menus (Core Principle)

Standing rule for the whole document: anywhere a feature looks like "pick from this list," the correct version is "write your own function/Type and hand it to the same slot." Any list that exists should only be pre-shipped examples, never the ceiling.

Corrected logic-variant search — takes real, user-authored candidates, never a fixed preset list:

```
type MyIdeaA extends VideoDiffusion { override fn
forward(frame) -> next_frame { /* any idea at all */ } }
type MyIdeaB extends VideoDiffusion { override fn
forward(frame) -> next_frame { /* a different idea */ } }

qai.search_logic_variants(base: VideoDiffusion, candidates:
[MyIdeaA, MyIdeaB], evaluate_on: data, metric:
"consistency_score")
```

Open event/hook system — authors invent their own named hook points, nothing predefined:

```
type MyDiffusion extends Diffusion {
  fn forward(input) -> output {
    let midpoint = compute_something(input)
    emit("midpoint_reached", midpoint)          //
    author-invented event name
    return finish(midpoint)
  }
}
model.on("midpoint_reached", fn(data) { ... }) //
anyone downstream can subscribe
```

Open custom checks/conditions — arbitrary user functions, not a menu of check types:

```
model.add_check(fn(state) { return state.loss <
previous_epoch_loss * 2 })
```

Further open extensions:

- User-defined composition operators: `qai.compose(logic_a, logic_b, combinator: fn(out_a, out_b) { /* your own blending idea */ })`
- Parameterized logic templates: `type Flexible extends Diffusion { fn forward(input, lookback: int) { ... } }`
- Live logic hot-swapping on a served model:
`model.swap_logic(new_logic, zero_downtime: true)`

- Community hook/event-name registry — lets common hook names converge by convention without the framework forcing it

71. Understanding Model Logic (Plain Definitions)

Logic = the actual steps a model follows to turn input into output — not the learned numbers (weights/parameters) and not the settings (hyperparameters), just the plan of action itself. Weights are closer to the neurons and their connection strengths (the hardware); logic is closer to the pattern of thinking that uses them (the process) — a useful, if simplified, analogy.

Per-modality examples of what "logic" plainly means:

- **Text:** "Look at the words so far. Guess the most likely next word. Say it. Repeat."
- **Video:** "Look at the current frame. Predict the next frame. Check it matches the character's appearance. Output it. Repeat."
- **Audio:** "Look at the sound so far. Predict the next tiny slice. Smooth it so it doesn't click. Add it. Repeat."
- **Image (diffusion):** "Start with noise. Guess what noise to remove. Remove a little. Repeat until clear."

Real model logic contains genuine control flow — loops, if-statements, boolean conditions — not simplified one-liners:

```
fn forward(input) {  
    let x = embed(input)  
    for layer in self.layers {  
        x = attention(x)  
        x = x + residual(x)  
        if self.use_normalization { x = normalize(x) }  
    }  
    if confidence(x) > 0.95 { return x }    // early exit if  
    already confident  
    return final_layer(x)  
}
```

Real precedent: MoE routing (`if gate_score(expert) > threshold: send_to(expert)`) is a genuine if-statement inside real production model logic (Mixtral/DeepSeek-style models).

Reality check on adoption: most AI engineers never touch logic — they use an existing architecture and tune settings/data. Logic-writing is what researchers do when inventing new architectures. Quantum's `forward() / override` system exists to make that second, smaller group's work far more accessible — not to replace the first group's simpler, and equally valid, workflow.

72. Two-Tier Logic System: Deep Logic vs. Composition Logic

Level 1 — Deep Logic: rewriting the actual math inside a Type's core methods.

```
type Regression { fn forward(x) { return dot(w, x) + b } }  
// a straight line  
type PolynomialRegression extends Regression {  
  override fn forward(x) { return dot(w, x^2) + dot(w2, x)  
+ b } // genuinely different math — a curve  
}
```

Level 2 — Composition Logic: wiring a model's already-exposed methods together into new behavior, with zero math required — the more accessible, much more commonly useful tier:

```
let frame = model.predict_next_frame(current_frame)  
if model.penalty(frame) > threshold { frame =  
  model.predict_next_frame(current_frame, retry: true) }  
model.output(frame)  
model.input(frame) // feeding output back into input — new  
behavior from existing pieces alone
```

Combining both — train a model until it internalizes a composed procedure, so the scripted version is no longer needed at inference time:

```

fn generate_long_video(model, target_duration) {
  let output = []
  let current = model.generate_first_chunk()
  while output.total_duration() < target_duration {
    let next = model.predict_next_chunk(current,
duration: 3s)
    if model.consistency_score(current, next) < 0.8 {
      next = model.predict_next_chunk(current,
duration: 3s, retry: true)
    }
    output.append(next)
    current = next
  }
  return output
}

model.train(
  scripted_procedure: generate_long_video,
  until: fn(result) { result.consistency_score > 0.95 &&
result.duration >= target_duration }
)
// after training, the model naturally does this – no
scripted retry loop needed at inference time

```

This maps to real research concepts: process supervision (training toward a defined multi-step success condition) and distilling a scripted procedure into an end-to-end model (self-play systems like AlphaGo use the same basic shape).

Honest caveat, kept as a standing note: making a technique easy to express doesn't make the underlying problem easy — long-horizon consistency (e.g. a 3-hour consistent video) remains a genuinely hard open research problem regardless of how well-designed the training loop is. The framework makes the technique accessible; it doesn't make the technique magic.

Supporting tools:

- `model.landing_progress()` — how close training is to reliably satisfying the `until` condition
- Partial-credit tracking — shows where a near-miss result degrades, not just pass/fail

- `model.compare_to_scripted(procedure)` — how the model's now-natural behavior differs from the original manual version

73. Full Multi-Model Composed Logic Example (Worked)

```
let story_model = qai.build(type: "transformer")
let scene_splitter = qai.build(type: "transformer")
let scene_generator = qai.build(type: "diffusion",
output_type: "video")
let validator_model = qai.build(type: "classifier")
let combiner_model = qai.build(type: "diffusion",
output_type: "video")

fn generate_full_movie(topic, target_duration) {
  let story = story_model.generate(topic)
  let scenes = scene_splitter.generate(story)
  let good_scenes = []

  for scene in scenes {
    if qai.detect_hardware().ram_available < 4.gb {
      var attempt = scene_generator.generate(scene,
duration: 3s, quality: "low")
    } else {
      var attempt = scene_generator.generate(scene,
duration: 3s, quality: "high")
    }
    while validator_model.check(attempt, previous:
good_scenes.last()) == "not matching" {
      attempt = scene_generator.generate(scene,
duration: 3s, retry: true)
    }
    good_scenes.append(attempt)
  }
  return combiner_model.combine(good_scenes)
}

model.set_execution_mode("sequential" | "parallel") //
process scenes one at a time or concurrently

// train the combiner specifically to merge/polish rough
```

```

outputs into a clean final result
let combiner_training_data = qai.dataset.create()
for run in past_generation_runs {
  combiner_training_data.add(input: run.raw_scenes, output:
  run.best_final_edit) }
combiner_model.train(combiner_training_data)

// bundle the whole thing – models + composed logic – as one
exportable unit
qai.package_pipeline([story_model, scene_splitter,
scene_generator, validator_model, combiner_model],
  logic: generate_full_movie
).export("full_movie_pipeline.qmodel")

```

- `qai.estimate_pipeline(generate_full_movie, target_duration: 3.hours)` – forecasts total compute time/cost before running
- Partial-run resume – continue a long pipeline from where it stopped instead of restarting entirely
- Lightweight scoring functions as a cheaper alternative to a full validator model, offered alongside it
- Live pipeline visualization – shows which model is currently working, on which piece, during a long run

74. The `model.logic{}` Block, Full Method Reference & Modality Traits

Block syntax – entry and exit are mandatory, since a logic block with no defined start/end isn't checkable or meaningful:

```

import qai
let story_model = qai.build(type: "transformer")
let video_model = qai.build(type: "diffusion", output_type:
"video")

model.logic {
  input(prompt) //
ENTRY – required

  let story = story_model.generate(prompt)

```

```

    let video = video_model.generate(story)

    if video_model.penalty() > threshold {
        video_model.learning.penalty += 5 //
reaching into a technique-specific property from inside logic
    }

    if video.quality() > 0.9 {
        model.online_learn(from: video) //
self-improves from its own best outputs
    }

    output(video) //
EXIT – required
}

```

Using a logic block is entirely optional – the plain path (`qai.build()` → `train()` → done) needs no logic block at all and remains just as simple as ever.

Full method reference, three tiers:

General (every model):

```

model.train() .finetune() .test() .predict() .generate()
.input() .output()
model.loss() .accuracy() .parameters() .hyperparameters()
.weights()
model.save() / .load() .export() .explain() .visualize()
.help()
model.eval_mode() / .train_mode() // behavior differs
during training vs. real use for some techniques
model.to(device) / .device() .clone() .summary() .size()
model.freeze_layer(name) / .unfreeze_layer(name)
.checkpoint() / .rollback() .state()

```

Type-specific (recap, Section 46): `.formula()` (Regression),
`.filter_visualize()` (CNN), `.attention_map()` (Transformer),
`.expert_routing()` (MoE), `.denoise_steps()` (Diffusion) – plus deeper
editable internals:

```

Regression: .set_coefficient(i, value) .lock_intercept()
.add_feature() .remove_feature()

```

```

CNN:         .set_filter_weights(layer, values)
             .freeze_filter(i) .visualize_activation(layer)
Transformer: .set_attention_head_weight(head, value)
             .mask_token(pos) .set_top_p() .set_top_k()

```

Technique/learning-paradigm-specific (Section 67): `.reward` / `.penalty` (Reinforcement), `.source_model` / `.frozen_layers` (Transfer), `.generate_own_labels()` (Self-supervised) — all reachable and manipulable from inside a logic block.

Modality traits — a third, separate axis from Type: a model's declared input/output modality grants automatic methods regardless of its internal architecture:

```

// ANY video-output model gets these, whether built on
Diffusion, GAN, or anything else
model.frame_count() model.duration() model.fps()
model.extract_frame(n) model.frame_rate(30)

```

Multi-objective training — training on more than one paradigm at once, genuinely possible:

```

model.train(data, objectives: [
  { type: "supervised", weight: 0.7 },
  { type: "reinforcement", weight: 0.3 }
])

```

This mirrors real practice — e.g. RLHF-style training genuinely combines supervised pretraining with reinforcement learning layered on top.

Real-world tweaking without any logic at all (most common real AI development, worth keeping visible alongside the advanced system): a developer might just lower `temperature` from 0.7 to 0.3 for more predictable output, or move a classifier's decision threshold from 0.5 to 0.7 to reduce false positives — zero architecture change, zero logic, just a dial turned.

Supporting tools:

```

qai.compare_outputs([a.output(), b.output()]).show() //
instant simple side-by-side view, no HTML/CSS/JS needed

```

```

logic.help() //
lists everything reachable at this exact point in a logic
block
model.has_logic() //
quick check: plain model or custom logic attached?

```

`.explain()` on a model with custom logic also shows which part of the logic influenced a given output, not just which weights did.

75. Mutable Weights, Remote Training & Self-Modification Safety

Logic makes weights genuinely mutable, not permanently frozen after training — a real, valuable capability that needs an equally real safety net.

Remote-compute-triggered training from inside logic:

```

model.logic {
  input(new_data)
  let job = qai.connect_remote_compute("kaggle" | "colab" |
"own_cluster")
  let updated_weights = job.train(base: model, data:
new_data, wait: false)

  on job.complete {
    let diff = model.preview_weight_diff(updated_weights)
    // safety gate – review before accepting
    if diff.looks_safe() {
      model.apply_weights(updated_weights) }
    else { model.flag_for_review(diff) }
  }
  output(model.status())
}

```

Mutability is opt-in, never the default:

```

model.set_mutability("frozen" | "mutable") // "frozen" is
the safe default

```

Full weight-change audit trail:

```

model.weight_history() // → [{ time, changed_by,
diff_summary }, ...]
model.rollback_weights(to: "2026-08-20")
model.pin_as_stable("v4") // trusted
fallback target
model.set_mutation_limit(max_changes_per_day: 3) // caps
runaway self-modification

```

Training itself IS written as logic — no hidden separate training step:

```

let vm = qai.connect_vm(sandboxed: true)
model.logic {
  input(task)
  let code = model.generate(task)
  let result = vm.run(code)
  if result.tests_passed { model.reinforce(code, reward:
1.0) } else { model.reinforce(code, reward: -1.0) }
  output(result)
}

```

Imitation learning by recording real demonstrations:

```

let screen_feed = qai.input.connect_stream(source: "screen",
type: "video")
let input_feed = qai.input.connect_stream(source:
"mouse_and_keyboard")
model.logic {
  input(demonstration_session)
  while demonstration_session.active {
    model.record_demonstration(state:
screen_feed.capture(), action: input_feed.capture())
  }
  model.train(technique: "imitation_learning", data:
model.recorded_demonstrations())
  output(model.status())
}

```

(Data collection needs no Type declared yet — the Type is chosen only when training actually begins.)

Thinking loop — repeated internal reasoning before committing to a final answer, matching how modern reasoning models work:

```

model.logic {
  input(task)
  var thought = model.think(task)
  while model.confidence(thought) < 0.9 {
    thought = model.think(task, previous_thought:
thought)
  }
  output(thought)
}

```

Terminology note: repeated self-retraining on one's own gaps is **continual learning** / a **self-improvement loop** — not meta-learning, which specifically means "trained to adapt quickly to new, unseen tasks" (Section 66).

Whole-model freeze/unfreeze and capability-based skipping, from inside logic:

```

model.logic {
  input(prompt)
  let category = classifier_model.classify(prompt)
  if category != "video" { video_model.freeze() } else {
video_model.unfreeze(); output(video_model.generate(prompt));
return }
  output(text_model.generate(prompt))
}

```

Lazy loading adapted per receiving device, from inside logic:

```

model.logic {
  input(request)
  if qai.detect_hardware().ram_available < 4.gb {
model.set_loading_strategy("layer_stream") }
  output(model.generate(request))
}

```

76. Dynamic Region Loading & Smart Memory Triage

.qmodel confirmed as the answer to "our own format" — needed specifically because GGUF can't carry logic:

```
model.export("my_model.qmodel")          // bundles logic
AND weights together
model.export_weights_only("model.gguf")    // valid ONLY for
models using unmodified built-in logic
```

Dynamic, per-request weight region loading — distinct from the one-time, permanent `extract_capability()` (Section 63):

```
model.logic {
  input(prompt)
  let topic = classifier_model.classify(prompt)
  let active_region = model.load_region(topic) // only
loads the weight region relevant to THIS request
  output(active_region.generate(prompt))
}
```

Smart memory triage logic — relevance-based keep/archive/delete, avoiding re-processing full history every time:

```
model.logic {
  input(message)
  let relevance = model.memory.relevance(message)

  if relevance > 0.5 { model.memory.retrieval.add(message)
}
  else { qai.export_to_file(message, format: "markdown",
path: "archive.md") }

  if model.memory.retrieval.has_stale_entries() {
model.memory.retrieval.delete_stale() }

  let recent_context = model.memory.working.last(1) //
only the last exchange, not the whole history
  output(model.generate(message, context: recent_context))
}
```

- Auto-summarization as a middle ground —

```
model.memory.summarize_and_compress(older_than:
"10_messages"), condense instead of fully drop
```

- Live-adjustable relevance threshold —
`model.memory.set_relevance_threshold(0.6)`
 - Archived content stays searchable —
`model.memory.search_archive("markdown files")`
 - Memory-decision transparency —
`model.memory.explain_decision(message)`
-

77. Logic System: Capabilities and Honest Limits

Real, substantial wins — not just marketing:

- RAM waste — `load_region()` loading only relevant weights per request
- Storage waste — memory-triage logic avoids hoarding everything indefinitely
- The black-box/no-control problem — before logic, a model ran one fixed way with no way to say "output this," "retrain now," "skip this part"; logic genuinely removes that ceiling
- Automating iterative improvement — a written training loop can run, evaluate, and adjust without manual restarts

Confirmed behavior for models without logic attached:

```
model.export("plain_model.gguf") // no logic — behaves
                                exactly like a normal, non-Quantum model, nothing lost
model.accuracy()                 // testing/inspection
methods work with zero logic block, always have
```

A downloaded third-party model (e.g. an open-weight model from outside the ecosystem) arrives with no Quantum logic attached by default — plain fine-tuning works exactly as normal; logic is always something you *add*, never something a foreign model brings with it.

Honest limits — what logic does NOT automatically solve:

- **"Infinite loop = infinitely smarter" is not quite right.** An unattended self-retraining loop only genuinely improves the model if its evaluator is trustworthy and can't be gamed. **Reward hacking** — a model finding a

way to satisfy the check without actually improving (trivial, narrow, or overfit answers that technically pass) — is a real, well-documented failure mode, and it's easiest to miss precisely when no one is watching the loop.

- **Compute/cost is real and unbounded** without limits — "let it run forever getting better" also means "let it run forever costing money and energy."
- **Debugging gets genuinely harder** — a model that self-modified many times is much harder to reason about than a static one.
- **Reproducibility gets harder** — if weights keep changing via self-directed logic, "the exact model that produced this result" becomes a moving target, in tension with the reproducibility fingerprinting idea (Section 26).

Safety mechanisms that make the power trustworthy, not just powerful:

```
model.detect_reward_hacking() // periodically tests against
a protected golden eval set (Section 55) to catch gaming vs.
real progress
model.self_improve(pause_every: 100, require_human_review:
true) // long loops can't run forever unattended
model.session_summary() // plain-language account of
what changed and why, after a long self-directed run
```

Bottom line: the RAM/storage/automation wins are real and substantial. The self-improvement power needs budgets, audit trails, and golden-eval checks treated as load-bearing parts of the feature, not optional add-ons — this exact combination (self-modifying weights + unattended loops) is where real AI systems have historically gone wrong in ways that are hard to notice until later.

78. Weight Table View & Confirmed Skill/Logic Relationship

Weights organized as a readable, filterable table, not just raw numbers:

```
model.weights.as_table()
// | layer | neuron | value | concept (if known) |
model.weights.export_csv("weights.csv")
```

```
model.weights.filter(concept: "curved edges")
model.weights.sort_by("magnitude", top: 20)
```

Weight-to-concept translation (Sections 4/29) powers the concept column:

`model.weights.translate([0.803, -0.001, 0.42])` → plain-language description, derived from correlating activation patterns across real inputs.

Skills vs. logic — confirmed, complementary relationship: a skill is a capability; logic is the traffic controller deciding when/how strongly it's active.

```
model.logic {
  input(request)
  let category = classifier_model.classify(request)
  if category == "rigging" {
    model.attach_skill(rigging_skill) } else {
    model.detach_skill(rigging_skill) }
  if category == "rigging" && category == "animation" {
    model.set_skill_weight(rigging_skill, 0.6)
    model.set_skill_weight(animation_skill, 0.4)
  }
  output(model.generate(request))
}
```

Self-directed skill creation (needs Section 77's safety mechanisms — budgets, review gates — since it spends compute autonomously):

```
model.logic {
  input(request)
  if model.knowledge_map().has_gap(request.category) {
    let new_skill = qai.skill.train(base: model, data:
qai.dataset.stream("huggingface://" + request.category),
name: request.category)
    model.attach_skill(new_skill)
    model.log("self-created a new skill: " +
request.category)
  }
  output(model.generate(request))
}
```

- `model.skills_table()` — every attached skill, its gate weight, and firing frequency, in the same table style

- `model.weights.as_table().diff(previous_checkpoint)` — which rows changed after training, in readable table form
- Concept clustering view — group the weight table by concept rather than by physical layer
- `model.skill_logic_map()` — which parts of logic reference which skills, so removing/renaming a skill doesn't silently break dependent logic

79. Router Core & Predictive Loading

Correction worth keeping explicit: less RAM used does not automatically mean faster — it's normally a tradeoff. Loading weights on-demand from disk saves memory but adds latency; the real win is avoiding loading things that are never needed, not "less RAM = inherently faster."

A small, always-resident "router core" is required to know what to load before loading it:

```
let router = model.router_core() //
small, cheap, ALWAYS in RAM
let target_region = router.predict_needed_region(prompt)
let active = model.load_region(target_region)
// only the heavy weights load, based on the router's cheap
prediction
```

This mirrors how real MoE routing already works internally, generalized to whole-model region loading.

Predictive prefetching based on conversation trajectory:

```
model.predictive_prefetch(based_on: conversation_trajectory,
confidence_threshold: 0.7)
// loads the likely-next-topic region in the background while
the current response is still being read
```

Where `model.memory` actually lives — three different physical realities behind one API:

```

model.memory.weights      // disk normally; streamed into
                           RAM/VRAM only when actively needed (Section 40)
model.memory.working       // RAM only – the context window,
                           transient
model.memory.retrieval     // disk/external DB by default;
                           specific retrieved pieces pulled into RAM briefly

model.memory.location_report() // → { weights: "disk,
                                   streamed", working: "ram, 1.2k tokens", retrieval: "disk, 40k
                                   entries" }

```

Supporting tools:

- `router.accuracy_report()` — how often the router's region guess was actually correct
- `model.set_memory_strategy("max_speed" | "min_ram" | "balanced")` — an explicit, visible tradeoff dial
- `model.expect_slower_first_response()` — honest cold-start expectation, since prefetching needs trajectory history it doesn't have yet on the first request

80. Full Weight Blueprint & Raw Editing API

The complete blueprint of what gets saved:

```

{
  format_version: "1.0", model_type: "transformer",
  saved_at: "2026-08-28",
  layers: [
    { name: "layer1", shape: [512,512], dtype: "f32",
      values: [[...],[...]] },
    { name: "layer3.attention_head_2", shape: [64,64],
      dtype: "f32", values: [...] }
  ],
  metadata: { technique: "transformer", hyperparameters:
    {...}, training_data_hash: "..." }
}

```

Raw, unrestricted editing — any single value, anywhere:

```

model.weights.raw()
model.weights.get(layer: "layer3", index: [12,45])
model.weights.set(layer: "layer3", index: [12,45], value:
0.91)
model.weights.set_range(layer: "layer3", rows: [0,10], value:
0.0)

```

Full export/hand-edit/reload round trip, usable with external tools too:

```

model.weights.save_blueprint("my_model_weights.json")
model.weights.load_blueprint("my_model_weights.json",
validate: true)  // shape/dtype validated before loading –
default true

```

81. Logic's Dual Mechanism & Honest Agency Clarification

Logic can trigger both weight edits and logic swaps — two distinct mechanisms:

```

model.logic {
  input(x)
  if model.confidence(x) < 0.5 {
    model.weights.set(layer: "layer3", index: [12,45],
value: 0.91)    // MECHANISM A – change the numbers
  }
  if model.performance_report().shows_pattern("always wrong
on negatives") {
    model.swap_logic(ImprovedForwardLogic)
  // MECHANISM B – change the steps themselves
  }
  output(model.generate(x))
}

```

Honest clarification, worth keeping explicit in the docs: logic does not give a model self-awareness or self-control over its own weights. A model only ever produces outputs (like a confidence score); **the developer's pre-written logic** decides what to do in response to that output. It looks like self-directed behavior, but precisely: the model produces a signal, and logic —

written in advance by a person — reacts to it. This is a real and valuable capability; it is not the model gaining agency.

Weights: flat in memory, structured when accessed — both true, at different levels:

```
model.weights.raw_flat()    // the actual underlying flat
                             array (what hardware wants)
model.weights.as_table()    // the same data reshaped into
                             named/labeled structure (what a person wants)
```

Same relationship as a spreadsheet file (flat bytes on disk, rows/columns when opened).

Supporting tools:

- Change-source labeling — every change tagged with why it happened (`triggered_by: "low_confidence_rule"`), feeding `weight_history()`
- `model.correlate_logic_to_weights()` — which logic rules caused the most weight changes over time
- A dashboard panel plainly separating "the model's raw output" from "logic's decision based on that output"

82. Weight Editing Safety Toolkit

Raw weight editing is the single most powerful and most dangerous capability in the system — every tool here makes the risk visible and reversible before it becomes permanent.

```
let proposed_change = model.weights.propose_edit(layer:
"layer3", index: [12,45], value: 0.91)
proposed_change.impact_report()
proposed_change.approve() | proposed_change.reject()
// treat a weight change like a code review

model.weights.estimate_blast_radius(edit)
// which capabilities share this weight

let lab = model.weights.open_lab()
```

```

// full sandboxed copy, safe to break
lab.set(...); lab.test(); lab.promote_to_live()

qai.transplant_weights(from: model_a, region:
"coding_capability", to: model_b) // graft between
compatible models

model.weights.anomaly_scan()
// flags statistically weird values automatically

model.weights.provenance(layer: "layer3")
// "92% original training, 8% manually edited on 2026-08-20"

model.weights.lock(region: "core_values", against: "training"
| "manual_edit" | "both") // protect specific regions

model.weights.set(...)
model.weights.last_edit_impact()
// automatic before/after benchmark comparison

qai.apply_patch(model, patch:
"community/fix_negation_bug_v1") // community fixes,
like mods, not full retrains

model.weights.playback(from: "epoch_1", to: "epoch_100",
speed: "2x") // watch values evolve like a video, not a
static heatmap

```

83. Consistency Pass Notes (Corrections Made)

A cleanup pass was run across the document to catch drift from rapid, session-by-session additions:

- Section 68's logic-variant search example still used the old hardcoded `mutations: [...]` list — corrected to match Section 70's open, user-authored-candidates version.
- `model.learning_paradigm()` and `learning_technique:` were used for the same concept in different places — standardized on `learning_technique` everywhere (matches the `learning_technique x { }` declaration syntax).

- Confirmed `type` and `technique` are intentionally interchangeable keywords (Section 63/66) — both appear throughout by design, not by accident.

Settled naming reference, for anyone reading this doc start to end:

- `type / technique` — the model's architecture (Regression, CNN, Transformer, MoE...) — interchangeable
- `learning_technique` — how it learns (supervised, self-supervised, reinforcement...) — a separate axis from the above
- `Input / Output` — the only names used for entry/exit endpoints, at both training-data-prep time and inference time (the earlier "transform" and "receiver" wording was retired in Sections 44 and 60)
- `Skill` — a small attached adapter capability, distinct from a full `Type`
- `logic { }` — optional composition/control-flow block; never required for the plain path (`build → train → done`)

This section exists as a running note for future consistency passes — if new contradictions are spotted later, they belong here, with the correction made at the original section too (not just noted here).

84. Explicit Exits, Operation Feedback & Relational Weights (Open Research)

Logic never implicitly "knows" when to stop — every exit is an explicit statement:

```
model.logic {
  input(x)
  var attempts = 0
  while attempts < 10 {
    let result = model.generate(x)
    if result.quality() > 0.9 {
      output(result) // this line IS how it "knows" —
      an explicit statement, not inference
      return
    }
    attempts += 1
  }
}
```

```

    output(result)          // fallback exit, always
    reachable
}

```

This is exactly why hard iteration caps (Sections 69/77) are mandatory, not optional — nothing else stops a badly-written loop from never reaching an `output()` .

SQL-style operation confirmations — any state-changing call prints a plain summary of what happened, by default:

```

qai.transplant_weights(from: model_a, region:
"coding_capability", to: model)
// → "245 values changed. Region 'coding_capability'
transplanted from model_a to model at 2026-08-29 14:02:11."

model.finetune(data)
// → "Fine-tune complete. 1,200 weights updated across 3
layers. Accuracy: 0.87 → 0.91."

```

Variable naming = instance identity — no special mechanism needed, ordinary programming:

```

let code_model = qai.build(type: "transformer")
let image_model = qai.build(type: "diffusion")
code_model.train(coding_data)    // unambiguous — a method
call on this specific named instance

```

Re-running a script mid-debugging never forces a full retrain — `if_exists: "resume"` (Section 42) is exactly the mechanism:

```

let model = qai.build(type: "transformer", name: "my_model",
if_exists: "resume")
model.train(data)    // first run: trains fresh. later reruns:
resumes, doesn't retrain from zero
model.weights.get(layer: "layer3", index: [5])    // checking
a weight mid-script is just a normal method call

```

Relational/self-organizing weights — an honest, open research direction, not a solved feature. Today's models mostly do NOT cleanly separate concepts into distinct regions on their own — concepts overlap and tangle

by default (Section 78's superposition point). Getting weights to naturally organize into clean, mergeable "relationship regions" is a real, active, unsolved research area (**disentangled representation learning**). Worth attempting, building on Self-Organizing Maps and Graph Neural Networks (Section 32), but must be framed honestly as an attempt at an open problem, not a guarantee:

```
type RelationalWeights extends Technique {
  fn forward(input) { ... }
  fn organize_by_relationship() { /* attempts to cluster
related concepts physically closer together */ }
  fn on_new_information(fact) {
    if contradicts_existing(fact) {
      retrain_region(affected_region, fact) }
    else { merge_into_region(fact, nearest_region) }
  }
}
```

Autonomous research + self-retrain — genuinely buildable from existing pieces, but the highest-risk combination in the system (compounds autonomous data-gathering with autonomous weight change):

```
model.logic {
  input(topic)
  if model.knowledge_map().has_gap(topic) {
    let research = qai.web_search(topic)
    model.online_learn(from: research, budget: "$5",
require_review: true) // mandatory, not optional, for this
pattern
  }
  output(model.generate(topic))
}
```

85. Honest Assessment & Meta-Design Additions

Honest take on the overall design: the recurring open-interface pattern (Technique, Environment, Bridge, Renderer, Input/Output, logic hooks) reused consistently rather than accumulating special cases is the real strength — rarer than any single feature. The logic system specifically

differs from orchestration frameworks like LangGraph/AutoGPT/CrewAI in kind, not just degree: those orchestrate *around* a frozen model; Quantum's logic reaches *inside* it (weights, `forward()`, skills, retraining) — a genuine architectural difference, though unproven at production scale the way those tools are.

Python-framework-instead-of-a-language tradeoff (honest note, not a plan change): a Python framework would be faster to build and adopt, but gives up the two real differentiators — "faster than Python because compiled" and "the logic block is real language syntax." Since real compiler infrastructure already exists (v3.x), that tradeoff favors staying the course.

Three-model self-play system (coder / reviewer-penalizer / researcher-asker) — a direct, valid extension of Section 14's role system:

```
let model = qai.build("combined")
model.add_role("coder", type: "transformer", goal:
"write_code")
model.add_role("reviewer", type: "classifier", goal:
"penalize_or_reward")
model.add_role("researcher", type: "transformer", goal:
"generate_questions")
model.connect("researcher" -> "coder", channel: "question")
model.connect("coder" -> "reviewer", channel: "code")
model.connect("reviewer" -> "coder", channel:
"reward_or_penalty")
model.set_evaluator(fn(code) { return run_tests(code) &&
reviewer.check(code) })
model.train(mode: "self_play", rounds: 1000)
```

WeightEditorModel — grounded in a real, established research concept: **hypernetworks** (a network that generates/edits another network's weights):

```
type WeightEditorModel extends Technique {
  fn analyze(target_weights) -> insight { ... }
  fn propose_edit(target_weights, goal: "improve_accuracy")
-> new_weights { ... }
}
let editor = qai.build(type: "WeightEditorModel")
editor.train(on: many_past_weight_edits_and_their_outcomes)
let improved = editor.propose_edit(some_model.weights, goal:
"reduce_bias")
```

86. Full Automation Utility Library

```
qai.time.now() .wait(3s) .elapsed(since: t)
.schedule(every: "1h", run: fn({})) .schedule_once(at: t,
run: fn({}))
qai.time.timeout(after: 30s, run: fn({}), on_timeout: fn({}))
qai.date.today() .diff(a,b) .add(date, days: 7)
qai.timezone.convert(time, from, to)

qai.system.run_command("ls") .env(key) .set_env(key,value)
qai.system.file.read(path) .write(path,data) .delete(path)
.exists(path) .watch(path, on_change: fn({}))
qai.system.cpu_usage() .ram_usage() .disk_usage()
.gpu_usage() .battery_level()
qai.system.processes() .kill_process(pid)
.start_process(cmd) .network_status() .is_online()

qai.notify.send("msg", via: "email"|"sms"|"push")
qai.notify.on_event("loss_diverged", alert: "email")

qai.retry(times: 3, backoff: "exponential", fn({}))
qai.parallel([a,b,c]) qai.race([a,b]) qai.debounce(fn()
{}), wait: 500ms) qai.rate_limit(fn({}), max_per_minute: 60)

qai.csv.read/write qai.json.parse/stringify
qai.zip.compress/extract qai.pdf.read(path)

qai.persist.save(key,value) / .load(key) //
survives restarts
qai.workflow.checkpoint(name) / .resume(name) //
resume long automations exactly where stopped
```

- Event-driven triggers over pure loops: `qai.on_file_change(path, run: model.retrain)`, `qai.on_schedule("daily", run: model.check_for_gaps)`
- A built-in automation dashboard — one view of every currently-running scheduled job/watcher/trigger and its status
- `qai.dry_run(my_automation)` — simulate without executing, critical before trusting anything touching real files/weights/money

- Automation dependency graph — explicit `depends_on` instead of manual timing sequencing
- Cost/impact preview before first-time running any automation, not just AI training

87. Media Manipulation & Full Data Type Toolkit

Image/video, single item and dataset-wide batch, same methods either way:

```
qai.media.image.load("photo.jpg").grayscale().resize(256,256)
// ONE item
qai.dataset.batch_apply(video_dataset, operations: [
    .color_shift(hue:30), .saturation(1.3), .brightness(0.1),
    .contrast(1.2),
    .grayscale(), .sepia(), .invert_colors(),
    .flip("horizontal"|"vertical"), .rotate(90),
    .crop(region), .resize(w,h), .pad(border:20), .blur(2),
    .sharpen(), .denoise(),
    .remove_background(), .watermark("draft"),
    .strip_metadata()
])
// MANY items, same ops
```

Ties directly into dataset prep (Section 60): `data.on_input(fn(x){`
`x.image = x.image.grayscale().resize(256,256); return x } )`.

- Color palette matching across a dataset; style-consistency batch check;
`qai.dataset.audit_media()` for format/resolution inconsistencies
before training

Strings:

```
qai.data.string.upper().lower().title_case().capitalize()/
.trim().strip_whitespace().remove_special_chars()
.replace().split().join().pad().truncate().contains().s
tarts_with().ends_with().reverse()
.count_words().count_chars().normalize_unicode().remove_ac
cents().slugify()
```

Numbers:

```
qai.data.number.round().floor().ceil().clamp(min,max).normalize(range).standardize()
.to_percentage().to_currency().is_prime().is_even().gcd()
.lcm().random().random_seed()
.convert(from: "celsius", to: "fahrenheit")
```

Files/documents/databases:

```
qai.data.excel.read/write/.to_dataframe()
qai.data.database.query(sql).insert().update().delete().backup().restore()
qai.data.archive.zip/.unzip()    qai.data.log.parse(path, format)

let db = qai.connect("database", type:
"postgres"|"mysql"|"sqlite")
db.watch_table("orders", on_new_row: fn(row) {
model.process(row) })
```

Supporting ideas: `qai.data.auto_detect(raw_input)` guesses type on import; chained transforms work identically for single items or full datasets; `qai.data.number.from_string(x, strict: true)` errors clearly on bad conversion instead of silent garbage; `qai.data.validate(input, expected_type, range)` at the Input boundary (Section 60).

88. Networking, Connectors & Real-World Automation Patterns

```
qai.api.call(url, method, headers, auth)
.retry_on_failure(times, backoff) .cache_response(ttl)
.rate_limit(max_per_minute)
qai.websocket.connect(url, on_message: fn(data){})
qai.oauth.connect("google"|"slack"|"salesforce")
qai.orchestrate([call_a, call_b, call_c], mode:
"parallel"|"sequential")
```

```
Reliability: qai.api.check_health(service) before depending on it mid-
run; qai.oauth.expiry_warning(service); qai.on_failure(primary,
fallback: secondary) .
```

Connector priorities, grounded in real 2026 practice — enterprise AI agent automation in 2026 centers overwhelmingly on connecting once to existing tools (Google Workspace, Slack, Salesforce, HubSpot, CRMs, calendars) and then taking actions like drafting emails, logging tasks, and updating records; the most common, highest-value automations across the field are ticket triage, invoice processing, lead qualification/scoring, scheduling, and reporting.

```
qai.connect("database") .connect("google_sheets")
.connect("slack") .connect("email")
.connect("crm") .connect("calendar")
.connect("huggingface") .connect("kaggle")
.connect("colab")
```

Bundled, self-contained export tools — a model shouldn't need an external library just to produce a PDF once deployed:

```
model.export("assistant.qmodel", bundled_tools:
["create_pdf", "create_markdown", "create_spreadsheet", "run_pyth
on"])
```

Supporting ideas: pre-built automation templates for the dominant patterns (`qai.template("ticket_triage")`, `qai.template("invoice_processing")`, `qai.template("lead_qualification")`); human-in-the-loop as the visible default for business-facing/high-risk automations, matching current best practice of starting with one workflow and keeping a human in the loop before scaling; a dedicated audit log for business-facing automations; a connector-health dashboard.

89. API Key System & Output Chaining

```
let served_model = qai.serve(model, port: 8080)
let key = served_model.generate_api_key(name: "client_a",
```

```
permissions: ["read","generate"], rate_limit: "1000/day",
expires: "90d")
served_model.revoke_api_key(key)
served_model.rotate_api_key(key)
served_model.list_api_keys()
```

- Per-key scoping to specific capabilities/skills; usage-based billing hook (`served_model.on_usage(key, fn(call){ qai.bill(...) })`), ties to Section 55's marketplace); alert on unusual key usage; single-use/temporary keys.

`model.output()` as a real, chainable, inspectable object, not just a string:

```
let result = model.output()
  .format("json")
  .redact(pattern: "emails")
  .pipe_to(another_model)
  .generate()
  .output()
  .translate(to: "spanish")

result.provenance() // full chain of models/manipulations
that produced this final result
```

90. Peer-to-Peer, MCP Exposure, Open Tool System & Self-Architecting Logic

Standard client-server (the normal case):

```
let served = qai.serve(model, port: 8080, public_url: true)
```

True peer-to-peer (no central server; a relay only helps two sides find each other, same principle as video-call connection setup):

```
let peer_connection = qai.connect_peer(peer_id:
"other_users_model_id", via: "qai_relay")
peer_connection.send(data)
```

Exposing a served model as a tool other AI platforms can call:

```
qai.serve(model, port: 8080, expose_as: "mcp_server")
```

Open tool authoring — the same fork-and-protect, open-interface discipline as Technique/Bridge/Renderer:

```
tool MyCustomTool { fn execute(args) -> result { ... } }  
model.logic { input(request);  
  output(MyCustomTool.execute(request)) }  
qai new tool --from="ExampleTool" --name="MyCustomTool"  
qai.tool_registry.publish(MyCustomTool)
```

The single most powerful (and most dangerous) thing logic enables — an AI that designs new AIs when its current architecture isn't enough:

```
model.logic {  
  input(persistent_gap)  
  if  
    model.knowledge_map().gap_unsolvable_with_current_architecture(persistent_gap) {  
    let candidate_types =  
      qai.propose_new_architectures(problem: persistent_gap)  
    let results = qai.search_logic_variants(candidates: candidate_types, evaluate_on: relevant_data)  
    if results.best().significantly_better_than(model) {  
      model.swap_logic(results.best())  
      model.log("designed and adopted a new  
architecture for: " + persistent_gap)  
    }  
  }  
  output(model.generate(persistent_gap))  
}
```

This combines gap detection, automated architecture search (Section 68), logic swapping, and safety logging into one loop — genuinely adjacent to automated architecture research. **Must always pair with Section 77's safety mechanisms** (budget caps, mandatory human review before adopting a new architecture, reward-hacking checks) — the more powerful the capability, the more load-bearing that pairing becomes, not less.

91. User-Input Round-Trip & Staged Output Pipeline

Pausing for real human input/clarification mid-logic, Python- `input()` - style:

```
model.logic {
  let raw_input = qai.input.from_user("Ask me anything: ")
  let improved = model.improve_prompt(raw_input)
  let confirmed = qai.input.from_user("Did you mean: '" +
improved + "'? (yes/edit): ")
  let final_prompt = confirmed == "yes" ? improved :
confirmed
  input(final_prompt)
  output(model.generate(final_prompt))
}
```

Fully self-contained logic (no human pause) remains just as valid and is the more common case:

```
model.logic { input(prompt); let improved =
model.improve_prompt(prompt);
output(model.generate(improved)) }
```

Full staged output pipeline — output manipulation is rarely one step; here it is properly staged:

```
model.logic {
  input(prompt)
  var draft = model.generate(prompt)
  // 1. first draft
  while model.confidence(draft) < 0.8 { draft =
model.generate(prompt, previous_attempt: draft, instruction:
"improve this") } // 2. self-review loop
  let review = reviewer_model.check(draft)
  // 3. different-model critique
  if review.has_issues { draft = model.generate(prompt,
feedback: review.issues) }
  let final = draft.redact(pattern:
"emails").format("markdown") // 4. manipulate before
release
  .add_disclaimer_if(review.risk > 0.5).translate(to:
user.preferred_language)
```

```

    if final.risk_score() > threshold {
// 5. human approval gate, only when risky
        qai.output.request_approval(final, on_reject: fn() {
draft = model.generate(prompt, feedback: "human rejected") })
        }
        qai.output.log(draft, to: "internal_log.md")
// 6a. silent internal record
        qai.output.to_user(final)
// 6b. what the person actually sees
        output(final)
    }

```

The full `qai.output.*` primitive set:

```

qai.output.to_user(x)
qai.output.stream_to_user(chunk)          qai.output.log(x,
to: path)
qai.output.to_format(x, [json, markdown])
qai.output.request_approval(x, on_reject: fn(){})
qai.output.mask(x, pattern)              qai.output.diff(v1, v2)
qai.output.rollback_to(previous_version)
qai.output.gate(x, condition: fn(x){...})

```

Output feeding back as the next input – a real cycle, not just a straight line:

```

model.logic {
    input(prompt)
    var result = model.generate(prompt)
    while qai.output.needs_more_work(result) { result =
model.generate(input: result, instruction: "continue and
refine") }
    output(result)
}

```

Supporting ideas: output version history within one request
(`result.version_history()`); selective output routing by content
(`qai.output.route(result, condition, to:`
`"slack"|"email"|"dashboard")`); an output SLA timer
(`qai.output.max_time(5s, fallback: "...")`) forcing release rather
than indefinite waiting; A/B output serving across user segments.

92. Input/Output Naming — Settled

A rename to `received()` / `produced()` was considered (to resolve ambiguity between block-level `input()` / `output()` and a model's own `.input` / `.output`) and then **reversed** — ordinary dot-notation already disambiguates this without new vocabulary:

```
model.logic {
  input(prompt)           // bare call, no dot —
  clearly the BLOCK's own entry statement
  let x = story_model.input // dotted onto a named
  model — clearly THAT model's property
  let y = image_model.output // dotted onto a different
  model — clearly a DIFFERENT model's property
  output(result)          // bare call — clearly
  the BLOCK's exit statement
}
```

Settled: keep `input` / `output` everywhere. Fewer overloaded words disambiguated by context beats more narrowly-named vocabulary, matching the language's minimal-syntax goal.

Supporting additions:

- Compiler hint on genuine misuse: bare `input` / `output` used outside a `logic {}` block → `"input() is only valid inside a logic block — did you mean model.input?"`
 - `model.io_summary()` — one glance at a specific model's expected input type and produced output type
 - Optional custom block-marker naming for readability in long logic blocks: `logic(entry: "request", exit: "response") { request(prompt) ... response(result) }`
 - `qai.print()` — a genuinely separate, third thing: plain console display with no model semantics at all, with a `visible: true|false` flag for suppressing internal narration from the end user, and the same formatting-chain support as everything else (`qai.print(result.format("markdown"))`)
 - Standing to-do: audit `Skill`, `Bridge`, and `Environment` for similar bare-vs-dotted naming overlaps as part of a future consistency pass
-

93. A Fun Showcase Project & Logic Tooling

A growing digital companion — uses memory triage, self-directed skill creation, mutable weights, and export bundling together, for something engaging rather than professional:

```
let companion = qai.build(type: "transformer", name: "Pip")
companion.set_mutability("mutable")
companion.memory.set_relevance_threshold(0.5)

companion.logic {
  input(interaction)
  if companion.memory.relevance(interaction) > 0.5 {
    companion.memory.retrieval.add(interaction) }
  if
    companion.knowledge_map().recurring_topic(interaction.topic,
    times: 5) {
    let new_skill = qai.skill.train(base: companion,
    data: qai.dataset.stream("huggingface://" +
    interaction.topic), name: interaction.topic)
    companion.attach_skill(new_skill)
    companion.log("Pip got curious about " +
    interaction.topic + " and learned it!")
  }
  output(companion.generate(interaction, context:
    companion.memory.retrieval.relevant_to(interaction)))
}
companion.export("pip_v1.qmodel") // hand this file to a
friend — Pip keeps growing under their interactions too
```

Supporting tooling, useful generally beyond this one project:

- `companion.growth_chart()` — visual timeline combining weight-evolution playback (Section 82) with skill acquisitions and memory milestones
- `model.logic.narrate(session)` — turns a raw execution trace into a plain-English story ("Pip noticed you kept asking about chess, so it studied and got better at it")
- `companion.compare_to(earlier_version)` — plain-language personality/behavior comparison over time, not just a weight diff

- `qai.diverge_check([pip_friend_a, pip_friend_b])` — how differently the same starting companion grew under two different people
- A logic "recipes" gallery — complete working examples (coding agent, movie pipeline, companion) bundled with the framework as starting templates
- `qai.lint_logic(my_logic)` — flags overly tangled logic and suggests splitting into named sub-functions
- `model.logic.explain_to_self()` — the system summarizing, in plain words, what its own attached logic currently does

94. Automated Scientist & Advanced Logic Patterns

An automated research loop — real, testable hypotheses checked against real evidence, not imagination, with mandatory review before internalizing anything:

```
let scientist = qai.build(type: "transformer")
scientist.set_mutability("mutable")

scientist.logic {
  input(research_question)
  let hypothesis = scientist.generate(research_question,
mode: "hypothesis")
  let sim = qai.connect_vm(sandboxed: true) // or a
domain-specific simulator/proof-checker
  let test_result = sim.run(hypothesis.to_experiment())

  if test_result.supports(hypothesis) {
    scientist.online_learn(from: { research_question,
hypothesis, evidence: test_result }, require_review: true)
  } else {
    let revised = scientist.generate(research_question,
mode: "hypothesis", ruled_out: hypothesis)
    // loop, or hand a negative result to a human
researcher — a negative result is still real knowledge
  }
  output({ hypothesis, test_result, confidence:
```

```
scientist.confidence(hypothesis) })
}
```

Directly in line with real, current research on AI-assisted hypothesis generation/testing. The hard, honest part is the "evidence" step — ensuring the system can't fool itself or be fooled by a flawed test — exactly the reward-hacking risk from Section 77, with real-world stakes if conclusions are trusted unchecked.

Fast/slow dual-speed logic — a cheap reactive layer plus a slow deliberate layer engaged only when needed, mirroring real dual-process reasoning as an architectural pattern:

```
model.logic {
  input(x)
  let quick = fast_model.generate(x)
  if fast_model.confidence(quick) > 0.9 { output(quick);
return }
  output(slow_model.think(x))
}
```

Further novel patterns:

- Logic evolution via genetic combination — breed pieces of two decent logic variants into a new one across generations, rather than only flat side-by-side comparison (Section 69)
- A "logic consultant" model — reads someone else's logic block and suggests improvements, applying the framework reflexively to itself
- Cross-domain logic transplant — automatically adapt a working pattern from one field (e.g. the movie pipeline's retry-until-consistent loop) to a different domain (e.g. iterative code review) without manual rewriting
- Time-travel logic debugging — step backward through a long-running session's actual decision points, not just weight history — "why did it choose this branch three steps ago"

95. Memory System — Full Consolidated Reference

Memory is scattered across several earlier sections; this is the single, unified map. (Cross-references below point to where each piece was

originally designed — nothing duplicated, just indexed together.)

1. Three physical tiers (Section 41/79): `weights` (permanent, changes only via training), `working` (live context window, cleared per session), `retrieval` (external, searchable, fast to update without retraining).

2. Context-window mechanics (Section 53): sliding window, RoPE/position tracking, KV cache, attention distance, decay/fade at the window's edge.

3. Low-memory loading (Section 40): layer streaming, prefetching, disk-as-overflow, partial GPU/CPU split — how the `weights` tier survives constrained hardware.

4. Region-based dynamic loading (Section 76): loading only the relevant weight region per request via a router core.

5. Smart triage (Section 76): relevance scoring, archiving to external files, deletion.

6. Short-term / long-term / discard system (new, sits on top of the above):

```
model.logic {
  input(interaction)
  let relevance = model.memory.relevance(interaction)
  if relevance < 0.2 { model.memory.discard(interaction) }
  else if relevance < 0.6 {
    model.memory.short_term.add(interaction) }
  else { model.memory.long_term.add(interaction) }
  output(model.generate(interaction, context:
    model.memory.short_term.recent(3)))
}

model.memory.short_term.on_repeated_reference(fn(item,
times_referenced) {
  if times_referenced >= 3 { model.memory.promote(item, to:
"long_term") }
})
model.memory.short_term.set_decay(after: "1 hour" |
"end_of_session")
model.memory.long_term.set_decay("never")
model.memory.short_term.set_capacity(max_items: 20,
on_overflow: "drop_oldest" | "drop_least_relevant")
model.memory.long_term.set_capacity(max_items: 10000,
on_overflow: "archive_to_file")
```

```

model.memory.long_term.add(interaction, tags:
["preferences","personal"])
model.memory.search(query: "favorite color", tags:
["preferences"]) // searches BOTH tiers, filtered by tag

```

Attaching/detaching multiple memory types on one model — a real, developer-controlled choice, worked example:

```

let model = qai.build(type: "transformer")

model.memory.attach("short_term")           // enable
working-style, session-scoped memory
model.memory.attach("long_term")           // enable
persistent, retrieval-style memory
model.memory.attach("shared_pool", source:
qai.connect("shared_memory_db")) // join a memory pool
other models also use

model.logic {
  input(x)
  if x.needs_recent_context {
    model.memory.detach("long_term") } // temporarily disable a
tier for this exchange
  output(model.generate(x))
  model.memory.attach("long_term")
  // reattach afterward
}

```

Each tier is genuinely optional and independently toggleable — a model can run with zero memory attached (pure stateless prediction), just `short_term`, just `long_term`, or any combination, decided explicitly by the developer, not assumed.

New ideas (not yet elsewhere in the doc):

- Memory "mood" inheritance — emotional/context weight tagged at save-time carries forward on recall
- Cross-model shared long-term memory pool (shown above via `shared_pool`)
- `model.memory.long_term.export("memories.json")` — portable memory file, separable from weights (e.g. share a companion's personality/weights but let someone start with fresh memory)

- `model.memory.usage_report()` — what fraction of long-term memory is ever actually retrieved vs. sitting unused
- `model.memory.schedule_review(every: "1 week", fn(item) {...})` — periodic re-evaluation of old memories against current relevance
- `model.memory.overview()` — one command surfacing all six pieces' current state at once (window fill, loading strategy, active regions, short/long-term counts, retrieval size)
- A Studio panel (Section 93) showing all six pieces as one connected live diagram

96. Tokenizers & What Actually Comes With a Downloaded Model

Downloading a real model (e.g. from Hugging Face) never gets you just weights — it's a bundle of several distinct files, worth naming explicitly since "download a transformer" hides real complexity:

```
qai.load("huggingface://some-model")
// pulls down, as a bundle:
//   - weights (the actual learned numbers)
//   - tokenizer (rules for splitting text into tokens —
//   often its OWN separate trained artifact)
//   - vocabulary file (the full list of known
//   tokens/subwords)
//   - config (architecture details: layer count, hidden
//   size, etc. — needed to even load the weights correctly)
//   - special tokens (e.g. [START], [END], [PAD], [UNK] —
//   reserved markers the model expects)
//   - model card (a README-style doc: what it's for,
//   training data, known limitations)
```

Tokenizer, explicitly, as its own first-class object — not just a hidden detail:

```
let tokenizer = model.tokenizer()
tokenizer.encode("hello world")    // → token IDs, e.g.
[15339, 1917]
tokenizer.decode([15339, 1917])    // → "hello world"
```

```
tokenizer.vocab_size()           // how many distinct
tokens it knows
tokenizer.special_tokens()       // → { start: "<s>",
end: "</s>", pad: "<pad>", unknown: "<unk>" }
```

Why this matters practically: a tokenizer is trained *specifically alongside* its model — using a mismatched tokenizer with a model's weights silently produces garbage, since token ID 15339 means something completely different across two different tokenizers. Worth an explicit compatibility check:

```
gai.check_tokenizer_match(model, tokenizer)
// → "Match confirmed" or → "Warning: this tokenizer was not
trained with this model — results will be unreliable"
```

97. Example Projects — What People Could Actually Build

Concrete scenarios illustrating the range the framework is meant to cover, from tiny to ambitious:

- **A movie-night picker** — tiny classifier trained on your own past ratings, run locally, decides what to watch
- **A smart to-do sorter** — small model learning your personal priority patterns instead of hand-written sorting rules
- **A personal writing style checker** — fine-tuned on your own past writing, flags when a draft doesn't sound like you
- **A home device behavior model** — the thermostat example (Section 91), or a smart light that learns a household's actual patterns
- **A growing companion** — Pip (Section 93), memory + self-directed skill growth, exported and shared
- **A game NPC decision-maker** — small RL policy or hand-written behavior tree (Section 100-ish/Section on NPCs), exported as an engine plugin
- **A multi-stage creative pipeline** — the movie-production example (Section 73): story → scenes → validated video → polished combiner

- **A self-improving coding agent** — Section 84's worked example, execution-grounded, safety-gated
- **An automated research assistant** — Section 94's hypothesis-test-refine loop, for a narrow, well-defined domain
- **A tiny embedded classifier** — small enough to run on a microcontroller (Section 62), for a single physical device task

The throughline across all of them: the framework scales from "a few lines, a tiny model, an afternoon project" up to "a multi-model, self-modifying research system" — using the exact same core vocabulary (`build`, `train`, `logic`, `input` / `output`) at every size.

98. CI/CD, Diagnostics & Community Tooling

CI/CD-style automated testing gate on model changes:

```
qai.pipeline_ci(model,
    on: "every_finetune" | "every_logic_change",
    run: [model.snapshot_as_tests(), model.stress_test(),
model.check_endpoints()],
    block_deploy_if: "any_test_fails",
    bind_evidence_to: "config_hash",           //
    eval results cryptographically tied to the exact config that
    produced them
    reject_if: "config_changed_but_evidence_stale" //
    a changed training config with no recomputed eval gets
    refused outright, not silently trusted
)
```

Treats a model change like a code change — nothing ships until it passes the same checks every time, automatically. Binding evidence to a config hash means a fine-tuning gate can render automatically on review, and a stale evaluation can never be mistaken for a current one.

Standardized "nutrition label" — one-glance model summary:

```
model.nutrition_label()
// → Size: 4.2MB | Energy per 1000 inferences: 0.02 kWh |
Bias scan: passed |
```

```
// Data sources: 3 (2 licensed, 1 unverified) | Known
limitations: weak on dates post-2024
```

Conversational front-end over the model's own introspection methods:

```
model.ask_about_self("what happens if I give you an image?")
// → "I only accept text input based on my current schema –
you'd need to attach an image capability first"
```

Whole-project diffing — data, logic, weights, and config together, not four separate diffs:

```
qai.project_diff(version_a, version_b)
```

A full staging environment for logic changes, more thorough than the lightweight sandbox (Section 69):

```
let staging = model.clone_to_staging()
staging.logic { /* try a risky change here */ }
staging.run_full_test_suite()
if staging.passes() { model.promote_from(staging) }
```

Natural-language-guided debugging:

```
qai.debug_from_description("the model seems to ignore
negative sentences")
// → "Checking attention patterns on negation words... found:
'not' and 'never' get low attention weight in layer 4.
// Suggest: model.weights.filter(concept: 'negation') to
inspect further"
```

Community showcase for small/hobby projects — lighter and friendlier than the formal benchmark leaderboard (Section 25), matching the spirit of Section 97's example projects: a gallery where people post small builds ("I made a movie-night picker with Quantum") for others to browse and start from.

99. Numerical Correctness, Evaluation Integrity & Alignment Techniques

Automatic differentiation as the real fix for hand-written gradient bugs — the compiler derives `update()` from `forward()` / `objective()` directly, rather than requiring anyone to write backprop math by hand:

```
type MyTechnique {
  fn forward(x) { return dot(w, x) + b }
  fn objective(output, target) { return (output - target)^2
}
#[autodiff] // update() auto-derived — an entire bug
class (wrong chain-rule application) becomes structurally
impossible
}
qai.numerics.check_gradient(MyTechnique, test_case) //
finite-difference verification, feeds into validate{}
(Section 56)
model.train(data, gradient_clipping: "auto", stability_check:
true)
qai.numerics.detect_instability(training_run) //
live exploding/vanishing gradient detection
```

Numerics operate on the underlying tensor math in `forward()`, independent of schema field names — a custom `input_schema` / `output_schema` composes correctly with autodiff automatically, since the schema only defines shape, not the math itself.

Data leakage prevented structurally, not just by convention — `TestSet` is its own restricted type:

```
let (train_set, test_set) = qai.dataset.split(data, train:
0.8, test: 0.2)
test_set.evaluate(model) // ✅ allowed
test_set.train(model) // ❌ compile-time error —
TestSet has no train() method at all
qai.detect_leakage(train_set, test_set) // scans for
overlapping/near-duplicate examples
```

Alignment as a first-class post-training method, called after `build()` / `train()`, alongside `finetune()` — not a build-time argument:

```

let model = qai.build(type: "transformer",
learning_technique: "supervised")
model.train(data)
model.align(method: "rlhf", preference_data: pairs)
model.align(method: "dpo", preference_data: pairs)
model.align(method: "constitutional", principles: [...])
model.align(method: "adversarial_training", attack_data:
adversarial_examples)
model.align(method: "debiasing", protected_attributes: [...])
model.align(method: "refusal_tuning", refuse_on: [...])

```

Connects directly to existing measurement tools (bias scanner Section 26, adversarial stress-testing Section 26, guardrails Section 17) — `align()` is where a model is actually trained to internalize what those tools measure.

Standing documentation rule (binding from here forward): every new method introduced must carry a real use case, following the `qai.man()` four-part shape (meaning, impact, example, why it exists) — no method added just because it sounds plausible. A full retroactive audit of all earlier sections against this rule is real, valuable, and not yet done — treated as a distinct future cleanup pass, same category as Section 83's consistency audit.

Runtime portability, confirmed: the embedded runtime bundled in a `.qmodel` export is built from the same Rust → C toolchain as the language itself — small and self-contained by construction, so "any environment with enough RAM can run it" is an honest claim, not aspirational.

100. Feature Weighting & Hardware-Realistic Design Philosophy

Which schema field is the training target is already fully specified — via `output_schema` itself, not a separate mechanism:

```

let model = qai.build(
  input_schema { size: f64, bedrooms: int, location_score:
f64 }, // features
  output_schema { price: f64 }
)

```

```
// the target `objective()` measures against
)
```

Feature importance weighting — for when not all input columns should matter equally:

```
model.set_feature_weights({ "location_score": 2.0,
                             "bedrooms": 1.0, "size": 0.5 })
```

Confirmed design philosophy, worth reflecting in the docs' tone directly:

most real Quantum projects are small models on modest RAM, not GPU-heavy training — `auto_adapt: true` exists to quietly make this the smooth, unremarkable default case, with GPU/VRAM handling as the edge case for genuinely large models, not the assumed norm.

```
model.train(data: my_dataset, auto_adapt: true) // `data:`
= what to learn from (always required); `auto_adapt` = a
separate, optional hardware-handling flag — both coexist, not
either/or
```

101. CLI Model Manager & Standalone Runtime

System-wide model management, distinct from any in-code API — a real, standalone tool (like `docker images` or `pip list`, but for local `.qmodel` files):

```
qai models list           // scans the machine, shows NAME /
SIZE / TYPE / LAST TRAINED / STATUS
qai models info <name>    // full detail: schema,
accuracy, size, weight-change history summary
qai models chart <name>    // quick terminal-rendered
loss/accuracy chart, no full Studio needed
qai models remove <name>   qai models search --
type=classifier
qai models running        // shows which models have an
active logic loop / server running
qai models cleanup        // flags
old/untouched/duplicate models
qai models watch          // live-updating terminal
```

```
view during training
qai doctor models           // scans every local
model for corruption/tamper flags in one pass
```

Running an exported model with zero surrounding project/code — the runtime's own interactive interface:

```
qai run thermostat.qmodel
// auto-detects interface style from the model's
output_schema: free text → chat REPL; structured schema →
labeled prompt-per-field
qai run pip_companion.qmodel           //
conversational REPL
qai run thermostat.qmodel --verbose --explain // shows
confidence/reasoning inline
qai run movie_picker.qmodel --input="action,2020" //
one-shot mode, single prediction then exit
qai run thermostat.qmodel --log=session.md //
saves the full session
qai run thermostat.qmodel --studio
// launches the full graphical Studio instead of plain text
```

Prints an offline confirmation on start — matters for trust when handed a
`.qmodel` file by someone else.

Full interactive CLI experience — banner + menu for exploration, direct arguments/options for scripting:

```
$ qai
[ASCII banner] Quantum Runtime v1.0 — 4 local models found
1) Chat with a model 2) Browse local models 3) Connect
a model to a service
4) View model info/charts 5) Run diagnostics 6) Exit

$ qai run thermostat.qmodel --verbose --connect=slack //
ARGUMENT (which model) + OPTIONS (how/where)
```

Organizing related small models together:

```
qai group create "game_npcs" --
add=np_movement,npc_dialogue,building_placer
qai group run "game_npcs"
```

Supporting quality-of-life features: `qai --version`, tab-completion for model names, `~/.qai/config.yaml` for default settings, `qai run <model> --dry-run` (validates load/schema without a real prediction), colored output by default with `--plain` for scripts.

102. Runtime Bundling Choice & CLI-Based Service/MCP Connection

Bundling the runtime is an explicit choice at export time, not automatic — a real size/tradeoff decision:

```
model.export("my_model.qmodel", include_runtime: true)    //
// bigger, runs standalone anywhere with no Quantum installed
model.export("my_model.qmodel", include_runtime: false)
// smaller, assumes `qai` already installed on the receiving
// machine
```

`qai` prints the resulting file-size tradeoff upfront when `include_runtime: true` is chosen, so the cost is visible before committing.

Connecting an exported model to external systems directly from the terminal, no code required:

```
qai connect thermostat.qmodel --api
// → generates a live endpoint AND an API key together, in
// one step, printed ready to use:
//   "Serving at https://your-model.qai.app:8080
//   API key: sk-quantum-a8f3d92e1c...
//   Try it: curl -H 'Authorization: Bearer sk-quantum-...'
//   https://your-model.qai.app:8080/predict"

qai connect thermostat.qmodel --mcp --name="Home Thermostat
Control" // exposes it as a discoverable MCP tool for agent
platforms
qai list --mcp-enabled           // shows which local models
are currently MCP-exposed
qai disconnect thermostat.qmodel --mcp
qai connect --all --mcp         // expose every local
model as MCP tools at once
qai mcp status                   // confirms an exposed
```

```
model is actually reachable/responding, not just "the command ran"
```

103. Additional Training Strategies

```
model.train(data, strategy: "data_parallel")      // same
model copied across devices, results averaged
model.train(data, strategy: "tensor_parallel")    // one
layer's math split across devices, for layers too big for one
model.train(data, strategy: "pipeline_parallel")  //
different layers live on different devices, staged flow
model.train(data, strategy: "zero_offload")       //
shards optimizer state/gradients across devices, cuts per-
device memory

model.train(data, progressive: { start_size: 64, end_size:
256, grow_every: "10 epochs" }) // progressive resizing
model.train(data, swa: true, swa_start_epoch: 40)
// Stochastic Weight Averaging
qai.population_train(model, population_size: 8, evolve_every:
"5 epochs") // Population-Based
Training
model.predict(x, test_time_adapt: true)
// test-time adaptation, per-input
model.train(data, optimizer: "sam", base_optimizer: "adam")
// Sharpness-Aware Minimization
model.finetune(data, unfreeze_schedule: { epoch_5:
"last_layer", epoch_10: "last_3_layers", epoch_20: "all" })
// progressive unfreezing

model.train(
  data: large_dataset,
  strategy: "sequential_layers", // train one layer's
weights at a time, others stay on disk until their turn
  ram_budget: "16GB",
  offload_to: "disk"
)
```

Blind spots addressed:

- Cross-hardware reproducibility — `qai.numerics.set_determinism_mode("strict" | "fast")` — floating-point results can vary slightly across GPU models
- Fault tolerance — `model.train(strategy: "data_parallel", fault_tolerance: "auto_resume_from_last_checkpoint")` — recover from a node crash without losing all progress
- Energy-cost-aware scheduling — `qai.schedule_training(model, data, prefer_hours: "off_peak")`
- Network bandwidth awareness in distributed setups — often the real bottleneck, not compute; not yet surfaced/optimized for

104. Environment (VM) Configuration, Feedback & Two VM Tiers

Two distinct VM tiers, same underlying `Environment` interface (Section 34), different weight classes:

```
let sandbox = qai.connect_vm(type: "code_sandbox", language:
    "python") // lightweight, fast, narrow
let full_vm = qai.connect_vm(type: "full_container", os:
    "ubuntu", allow_network: true) // a real general-
    purpose computer
full_vm.install_package("numpy") full_vm.run_command("git
    clone ...") full_vm.create_file("report.txt", content)
full_vm.zip(files)
```

Configurable, whitelisted, enable/disable-able capabilities on any VM:

```
let vm = qai.connect_vm(
    sandboxed: true, language: "python",
    allow_network: false, allow_file_access: false,
    allow_subprocess: false,
    max_memory: "512MB", max_execution_time: "5s",
    allowed_imports: ["math", "json"], allowed_packages: []
)
vm.enable("network") vm.disable("file_access")
vm.current_permissions()
```

SQL-style operation feedback on every execution:

```
vm.run(code)
// → "Executed in 0.03s. Exit code: 0. Memory used: 12MB.
Output: '42'. No errors."
vm.run(bad_code)
// → "Failed after 0.01s. [WHAT] SyntaxError on line 3 [WHY]
Missing closing parenthesis [FIX] Check line 3"
```

RAM-safe sequential training queue, avoiding concurrent training overload:

```
qai.train_queue([model_a, model_b, model_c], mode:
"sequential")
```

Supporting ideas:

- `vm.run(code, on_quota_exceeded: "kill_and_report")` — explicit behavior on hitting resource limits
- `vm.detect_breakout_attempts()` — flags code specifically trying to exceed granted permissions
- `vm.snapshot("clean_state") / .restore("clean_state")` — reset to known-good state between runs
- `vm.run(code, max_output_size: "1MB")` — prevents runaway output from flooding memory/logs
- `vm.audit_log()` — full timestamped record of everything ever run through this VM instance

Blind spots: multi-VM resource contention (no fair-scheduling governance yet when several logic blocks share VM access concurrently); VM version/dependency drift affecting reproducibility across machines; VM execution time not yet folded into the cost-estimation tools (Section 12).

105. VM Realism, Project Integration & Developer Notebook

Corrected self-play syntax (fixes a drift from Section 14's established design — a self-play system uses roles/connections, not a bare

```
train(evaluator: fn) call):
```

```
let model = qai.build("combined")
model.add_role("coder", type: "transformer")
model.add_role("reviewer", type: "classifier")
model.connect("coder" -> "reviewer", channel: "code")
model.set_evaluator(fn(code) { return
  vm.run(code).tests_passed && reviewer.check(code) })
model.train(mode: "self_play", rounds: 1000)
```

Realistic VM mechanics – bootstrap, access-vs-competence, lazy install, persistence:

```
let vm = qai.connect_vm(type: "full_container", os: "ubuntu")
// first ever call: no container backend present →
// downloads/installs one (e.g. Firecracker) automatically,
// caches the base image
// every future call: instant, already present
```

Access ≠ competence – enabling VM capabilities gives a model access; using them well is a separate, trainable skill:

```
let vm_skill = qai.skill.train(base: model, data:
  qai.dataset.stream("huggingface://shell_command_examples"),
  name: "vm_navigation")
model.attach_skill(vm_skill)
```

Lazy, on-demand package installation – never eager, only installs what running code actually requires:

```
vm.run(code) // installs numpy automatically ONLY if the
              code imports it – no upfront install list needed
```

Persistent VM state across sessions:

```
vm.save_state() // installed packages, cached image,
                created files preserved
// next session picks up exactly where it left off
```

Backend choice, since different container backends have real tradeoffs:

```
qai.connect_vm(type: "full_container", backend: "firecracker"
| "docker" | "gvisor")
```

Project-folder integration – VM state lives inside the same project structure as everything else (Section 42):

```
project/
├─ models/   ├─ logic/   ├─ data/
├─ vm/                // persistent VM state:
packages, cached image, created files
├─ notebook/          // developer's own notes
(below)

qai.package_project("my_project") // bundles EVERYTHING,
including VM state, as one portable unit
```

Developer notebook – distinct from automated system logs; captures what the developer noticed/thought/decided, not what the system did:

```
project.notebook.add_note("Found the model ignores negation
words – investigating layer 4")
project.notebook.log_error(error, context: "during finetune
on 2026-08-30")
project.notebook.log_security_finding("VM allowed an
unexpected network call – tightening allowlist")
project.notebook.history()
project.notebook.search("negation")
```

106. Experiment Tracking Dashboard & Paged KV Cache Serving

Experiment tracking – confirmed via real 2026 industry research as the single most consistently prioritized MLOps capability ("the glue" of the ML lifecycle: a trustworthy record of what was trained, with which data and code, and what came out). The top real gap in the whole design, now fixed:

```
qai.experiments.track(model, run_name:
"regression_v3_lr0.01",
```

```

    tags: ["baseline", "learning_rate_sweep"],
    log: { hyperparameters: model.hyperparameters(),
data_version: dataset.version_hash(), code_version:
qai.git_hash() }
)

qai.experiments.dashboard()
// → sortable, filterable table of every run: name, tags,
hyperparameters, accuracy, loss, timestamp, linked
dataset/code version

qai.experiments.compare(["regression_v3_lr0.01",
"regression_v3_lr0.001"]) // side-by-side, any two or more
runs
qai.experiments.best(metric: "accuracy", filter: { tags:
"learning_rate_sweep" })

```

This ties together and gives a real home to results from `.train()`, `qai.compare_significance()` (Section 55), and `model.compare_logic()` (Section 69) — the dedicated surface for "what have I actually tried, and what worked," which nothing before this centralized.

Paged KV cache management for serving many simultaneous requests — distinct from Section 53's per-conversation KV cache handling, addressing real concurrent-load efficiency:

```

qai.serve(model, kv_cache_strategy: "paged")
// manages KV cache memory in small, reusable blocks shared
across ALL simultaneous requests,
// instead of each conversation reserving its own fixed-size
chunk — a real, named serving technique (PagedAttention-
style),
// not a per-conversation optimization

```

107. Operational Maturity Suite

Dataset versioning with real diffing, content-addressed, not just filenames:

```

dataset.commit("v4", message: "added 2000 new labeled
examples")
qai.dataset.diff("v3", "v4") // → exactly what rows/columns

```

```

changed, added, removed
model.train(data: dataset.at_version("v4")) //
reproducibility tied to an exact data snapshot

```

Labeling quality control, extending `qai.label_session()` (Section 61):

```

qai.label_session(data, quality_control: {
  gold_standard_questions: gold_set,      // seeded into
the batch to check labeler accuracy
  inter_annotator_agreement: true,        // measured
when multiple people label the same items
  flag_labeler_if_accuracy_below: 0.85
})

```

Aggregate cost/billing dashboard, beyond Section 12's per-run estimate:

```

qai.billing.dashboard(period: "this_month") // → total
spend across every training run, projected vs. budget
qai.billing.set_budget_alert(monthly: "$500")

```

Incident-management integration, extending `qai.notify()` (Section 86):

```

qai.notify.connect("pagerduty" | "opsgenie")
qai.notify.on_event("model_degraded_in_production",
escalate_to: "on_call_rotation")

```

Pre-experiment statistical power calculation, before running an A/B test, not just after (complements Section 55's `compare_significance`):

```

qai.calculate_required_sample_size(expected_effect_size:
0.05, confidence: 0.95)
// → "You need approximately 2,400 samples per variant to
reliably detect this difference"

```

Standardized, side-by-side model-card comparison, extending the single-model card (Section 61):

```

qai.compare_model_cards([model_a, model_b, model_c]) // one
standardized table: data sources, size, bias scan,
limitations, license

```

Marketplace discovery quality signals, for the technique/skill registry (Section 17):

```
qai.registry.rate(technique, stars: 5)
qai.registry.browse(sort_by: "most_used_this_month" |
"highest_rated")
```

Fine-grained dataset governance, distinct from model-level roles (Section 89):

```
qai.dataset.share(dataset, team: ["alice","bob"], role:
"viewer" | "editor" | "owner")
```

108. The Learning Curriculum (Finally Documented)

The framework's actual "learn it in about a week" path — previously only discussed, now recorded as the real reference:

1. **Language basics** — variables, functions, control flow, the `Time / System` utilities (Section 86)
2. **Your first model** — `build` → `train` → `chat`, zero theory
3. **Seeing inside** — `.visualize()`, `.explain()`, the dashboard
4. **Improving a model** — `.finetune()`, gaps, weaknesses
5. **Composition** — two models, `.pipe_to()`, Bundles
6. **Logic fundamentals** — one `if`, one loop, `input()` / `output()`
7. **Data & automation** — datasets, the automation utilities
8. **Advanced logic** — self-play, mutable weights, skills
9. **A real project** — build something complete end to end (e.g. the sentiment classifier, Section 13, or Pip, Section 93)
10. **Deployment** — `serve()`, `export()`, API keys

Each module is independently completable — stopping after Module 5 already leaves someone with a genuinely useful, complete skill set, not a sense of having failed to finish.

109. Generation Sampling, Model Loading & Tokenizer Details

```
model.generate(prompt, top_p: 0.9, top_k: 50,
               repetition_penalty: 1.2, frequency_penalty: 0.5,
               presence_penalty: 0.3, do_sample: true, seed: 42)

model.generate(prompt, max_new_tokens: 200)  // limits ONLY
newly generated text
model.generate(prompt, max_length: 200)      // limits
prompt + generated text COMBINED – an important, easily
confused distinction

tokenizer.encode(text, padding_side: "left"|"right",
                 truncation_side: "left"|"right", add_special_tokens: true)

qai.load("huggingface://some-model",
        revision: "a1b2c3d", trust_remote_code: false,  //
security-relevant: refuses to execute bundled custom code
unless explicitly allowed (Section 33)
        local_files_only: false, cache_dir: "~/.qai/cache",
        device_map: "auto")

model.train(data,
            eval_steps: 500, save_steps: 500, logging_steps: 100,
            dataloader_num_workers: 4, resume_from_checkpoint: true,
            max_grad_norm: 1.0, adam_beta1: 0.9, adam_beta2: 0.999,
            adam_epsilon: 1e-8,
            class_weights: {"rare_class": 3.0})
```

Notably important, not just cosmetic: sampling controls (used in essentially every real text-generation call) and `trust_remote_code` (a genuine, documented security concern with downloaded models).

110. Checkpoint Management, Training Mechanics & Data Handling

```
model.train(data, save_total_limit: 3,
            load_best_model_at_end: true, metric_for_best_model:
```

```

"eval_accuracy")

model.train(data, weight_decay: 0.01, weight_decay_exclude:
["bias", "layer_norm"]) // applying decay to bias/norm
params often hurts training

tokenizer.set_pad_token_if_missing("<pad>") // common
gotcha – some tokenizers have no default pad token
tokenizer.pad(dynamic: true) // pad each
batch to its own longest example, not a fixed length

qai.dataset.stream(source, shuffle_buffer_size: 10000) //
rolling-buffer shuffle approximation for infinite streams

model.train(data, on_nan_loss: "skip_batch" |
"stop_training")

model.train(data, batch_size: 8, gradient_accumulation_steps:
4, num_devices: 2)
model.effective_batch_size() // → 64, shown explicitly
since it's easy to miscalculate

qai.load("huggingface://some-model",
attention_implementation: "flash_attention_2|"eager",
torch_dtype: "bf16")
qai.load(model_path, ignore_mismatched_sizes: true) // for
loading a pretrained model with a changed output head

```

Notably important: weight-decay exclusion and the missing-pad-token issue are common, real causes of subtly-worse training that people often only learn about the hard way – worth flagging in docs as common-mistake warnings, not just optional flags.

111. Reproducibility, Splitting & Sharing Details

```

model.train(data, early_stopping_patience: 3,
early_stopping_min_delta: 0.001)
model.train(data, warmup_ratio: 0.1) // first 10% of total
steps, regardless of dataset size

model.set_label_mapping(label2id:

```

```

{"positive":0,"negative":1}, id2label:
{"0":"positive",1:"negative"}) // keeps labels in sync
everywhere, including .explain()

qai.dataset.split(data, train: 0.8, test: 0.2, stratify_by:
"label") // preserves class balance, avoids a common
random-split mistake

qai.set_global_seed(42) // one seed covering data
shuffling, weight init, dropout – not just the model
qai.set_log_level("silent"|"warning"|"info"|"debug")

model.push_to_registry("my-username/my-model", private:
false)

qai.dataset.load("huge_file.csv", mmap: true) // reads on
demand from disk, never loads the whole file into memory

```

Notably important: stratified splitting and the global seed – an unstratified split silently skewing results, and "reproducible except the data shuffle used a different seed," are both classic, frequent real bugs.

112. Class Balance, Evaluation Metrics & Learning-Rate Tuning

```

qai.dataset.check_class_balance(data) // → "Class 'fraud':
0.3% of data – severely imbalanced"
data.balance(method: "oversample"|"undersample"|"smote")

model.confusion_matrix(normalize: "true"|"pred"|"all"|"none")
model.accuracy(top_k: 5) // "was the right answer in the
top 5 guesses"

qai.find_learning_rate(model, data) // trains briefly
across a rate range, plots loss vs. rate, suggests the
steepest-descent point

model.train(data, gradient_accumulation_steps: 4,
scale_loss_by_accumulation: true) // prevents an
accidentally too-aggressive effective batch

```

```

model.export(..., pin_environment: true) // records exact
qai/runtime version, catches "works on my machine" failures
later

model.generate(prompt, stop_sequences: ["\n\n", "END", "###"])

model.predict_batch(inputs, batch_size: "auto") // probes
memory, picks the largest safe batch size itself

```

Notably important: the learning-rate finder — guessing a learning rate is one of the most common early-training mistakes, and this removes a frequent source of wasted runs.

113. Loss Functions, Per-Layer Rates & Inference Optimization

```

model.train(data, loss_fn: "cross_entropy", label_smoothing:
0.1)
model.train(data, loss_fn: "focal_loss", focal_gamma: 2.0)
// for severe class imbalance, focuses on hard examples

model.train(data, layer_lr: { "embedding": 0.0001,
"classifier_head": 0.001 }) // different parts learn at
different speeds

model.train(data, metric_smoothing: "ema", ema_decay: 0.9)
// smooths noisy epoch-to-epoch validation fluctuation

qai.numerics.set_cuda_deterministic(true) // slower, but
bit-for-bit reproducible on the same hardware (ties to
Section 103)

model.tokenizer.trim_vocab(keep_tokens_used_in: my_dataset)
// shrink vocabulary to only what's actually needed

model.predict_batch(inputs, preserve_order: true) //
guarantees output[i] matches input[i] even if processed out
of order internally

qai.inference_mode {
  let result = model.predict(x) // skips gradient

```

```
tracking entirely – faster, less memory, pure-inference code
}
```

Notably important: `inference_mode` – running predictions without gradient tracking is one of the most common real production performance wins, and its absence would be a genuine practical gap.

114. Hardware-Burned Export: Chip Detection, Types & Reversibility

Most AI models never change weights after training – that's the normal, universal default everywhere, including most Quantum models. Mutable weights via `logic` (Section 75) are a deliberate, opt-in exception, not the norm, and this section's chip-export options sit at the far end of "permanent" on that same spectrum.

Chip detection and honest capability reporting:

```
qai.detect_chip()
// → { type: "flash", model: "ESP32-S3", storage_available:
      "4MB", supports_erase: true }
```

Export targets, by real reversibility:

```
model.export(target: "chip", chip_type: "flash")    //
erasable/rewritable (like an ESP32, Coral Edge TPU, Pi Pico)
– same technology as a USB drive or SD card, just built into
the device
model.export(target: "chip", chip_type: "fpga")      //
reconfigurable at the circuit-logic level, not just weights –
most flexible, more costly/power-hungry
model.export(target: "chip", chip_type: "asic")      //
fabricated permanently into silicon – genuinely irreversible,
no erase option ever offered for this type
```

`qai.chip.erase()` / `qai.chip.reflash(new_model)` are only ever offered when `supports_erase: true` – the interface itself reflects hardware truth rather than exposing an option that would silently fail on permanent silicon.

Hybrid path — iterate on flash, only "graduate" to permanent ASIC once genuinely stable:

```
model.chip_readiness_check()  
// → "This model has changed 12 times in the last 30 days –  
NOT recommended for permanent ASIC burning yet.  
//   Recommend flash deployment until stability improves."
```

ASIC/fixed-function burning is a real, valuable, narrow option specifically for a model that is final and never needs to change — a fixed edge-device task (a doorbell's person-detector) — and is fundamentally incompatible with anything using `logic`, mutable weights, skills, or online learning.

115. Strategic Critique — Standing Notes

Captured here as durable project notes, not features to build, since these are risks about the *process* of building this, not gaps in the design itself:

- **API surface size** — 113+ sections is more than any one person can hold in their head; discoverability at this scale needs either a much smaller, aggressively-surfaced "80% of real use" core, or genuinely strong search — neither is stress-tested yet.
- **Unverified core claims** — "faster than Python," "compiles to C," "5 lines instead of 100" are the actual value proposition and have never been benchmarked against anything real.
- **Untested feature interactions** — mutable weights + distributed training + self-modifying logic + dynamic region loading were each designed individually; how they behave *together*, under real concurrent load, has never been examined.
- **No defined MVP** — the single most important missing artifact is not a feature, it's a written answer to "what ships first, by when."
- **No named primary audience** — total beginners and self-modifying-architecture researchers have contradictory needs; v1 needs to pick one.
- **No adoption plan for the first real users**, independent of the marketplace/registry tooling that assumes a community already exists.

- **Novel vs. valuable, conflated** — a meaningful share of the design is a clean recombination of existing ideas (fine — most good engineering is), but the genuinely differentiated core (open `Technique / Type` interface discipline, logic as a real language construct, mutable weights via logic) deserves to stay visible as distinct from the rest.
- **No continuity plan** — what state would the project be in if development paused for six months; is there enough decision-level documentation (not just feature documentation) for someone else to pick it up.

116. Core Method Glossary — Completeness Pass

A systematic pass identifying the ~30 most frequently-used methods across the entire document and giving each a full definition, following the standing documentation rule (Section 99). These are the load-bearing methods — nearly everything else in the design calls one of these underneath.

`gai.build(type, learning_technique, ...)` — creates a new model. MEANS: instantiates a Type with (optionally) a learning technique and schema. ARGUMENTS: `type` (required — architecture, e.g. "regression"), `learning_technique` (optional — how it learns, defaults sensibly per type, Section 66), `input_schema / output_schema` (optional — declared data shape, Section 91), `name / if_exists` (optional — for resumable projects, Section 42). RETURNS: an untrained model object with random/default weights.

`model.train(data, ...)` — teaches the model from data. MEANS: runs the actual learning process. ARGUMENTS: `data` (the dataset, or `environment`: for reinforcement learning instead), `epochs / target / goal` (stopping criteria), `auto_adapt` (hardware-aware, Section 100), `on_step` (per-step callback, Section 105). Continues from current weights — never resets a model that already has some.

`model.generate(prompt, ...)` — produces output for generative models specifically. MEANS: runs a forward pass and formats a human-facing result. ARGUMENTS: sampling controls (`temperature` , `top_p` , `top_k` , Section 109), `max_new_tokens / max_length` , `stop_sequences` . Distinct

from `predict()` — `generate()` implies open-ended output; `predict()` implies a structured schema-shaped answer.

`model.predict(input)` — produces a structured prediction. MEANS: forward pass, result matches `output_schema` exactly, field-accessible by name (Section 92). Available on every model type, whether generative or not.

`qai.connect(service | vm | environment | peer, ...)` — establishes a connection to something external. MEANS: one word, several real targets — a database, a VM/Environment (Section 34), a cloud platform, or another Quantum instance peer-to-peer (Section 90). The specific target and its own arguments determine behavior; `qai.connect("database", type: "postgres")` differs meaningfully from `qai.connect_peer(peer_id, via: "relay")`.

`model.export(path, ...)` — packages a model for use outside the current project. MEANS: writes weights (and optionally logic, runtime, licensing) to a portable file. ARGUMENTS: `include_runtime` (Section 102), `include_training_logic` (Section 105), `target` (platform-specific, e.g. "unity"), `license`, `pin_environment` (Section 112).

`qai.load(source, ...)` — brings an existing model/dataset into the current project. MEANS: reads from a local file, a streamed remote source, or a marketplace registry. ARGUMENTS: `revision` (pin an exact version), `trust_remote_code` (security-relevant, Section 109), `local_files_only`, `device_map`.

`model.add_layer(name, type, ...)` — adds a named computational layer to a model's architecture. MEANS: extends the structure before training. Every layer type (embedding, attention, classifier, moe) has its own relevant parameters (Section 46).

`qai.dataset.stream(source, ...)` — pulls data incrementally rather than downloading it whole. MEANS: avoids full-dataset storage cost, useful for huge or remote datasets (Section 12). ARGUMENTS: `batch_size`, `shuffle_buffer_size` (Section 110).

`qai.serve(model, ...)` — makes a model callable as a live network service. MEANS: starts a server; requests match `input_schema`, responses

`match output_schema` (Section 102). ARGUMENTS: `port`, `protocol` (`rest/grpc/websocket/graphql`), `auth`, `rate_limit`.

`model.help()` — self-documentation. MEANS: prints exactly what this specific model instance supports right now — its methods, parameters, and a runnable example (Section 21). Different from `qai.man(topic)`, which documents a general concept independent of any one model.

`model.finetune(data, ...)` — improves an already-trained model without retraining from zero. MEANS: adjusts existing weights toward new data/skill. ARGUMENTS: `mode` (`adapter/full/head-only/lora/prompt-tuning/distillation`, Section 5), `focus` (target a specific weak area, Section 43).

`vm.run(code)` / `environment.run(action)` — executes something in a sandboxed or real environment. MEANS: the actual "doing" step of execution-grounded learning (Section 34) or agentic tool use. Returns a real result object (`tests_passed`, `error`, `output`), not just a boolean.

`logic { output(x) }` — the exit statement of a `logic{}` block. MEANS: hands back the block's result; execution of that block ends here (Section 74). Never implicit — always an explicit statement (Section 84).

`model.explain(prediction)` — produces a human-readable rationale for a specific output. MEANS: surfaces which inputs/weights most influenced this one result (Section 4). Distinct from `model.weights.translate()`, which explains a raw weight value in isolation, not a specific decision.

`qai.connect_vm(...)` — establishes an Environment specifically for code execution (Section 34/104). ARGUMENTS: `type` (`code_sandbox/full_container`), `sandboxed`, `allow_network` / `allow_file_access` / `allow_subprocess`, `max_memory` / `max_execution_time`, `allowed_imports`.

`data.add(...)` / `dataset.add_row(...)` — inserts a record into a dataset object. MEANS: builds or extends training data programmatically (Section 90/97), as an alternative to loading from a file.

`model.pipe_to(other_model)` — chains one model's output into another's input. MEANS: the basic building block of multi-model pipelines (Section 37) — a compatibility check runs automatically before the data actually flows.

model.set(...) — a general-purpose setter for named parameters.

MEANS: adjusts a hyperparameter or configuration value by name (e.g.

model.set(learning_rate: 0.001)) — the generic counterpart to more specific setters like **set_feature_weights()**.

model.merge(other_model, ...) / **qai.merge_into_regions(...)** —

combines two or more models. MEANS: two distinct operations sharing a name-root — **merge()** bridges two live, still-separate models (Section 10); **merge_into_regions()** fuses them into one object with named, addressable parts (Section 96). The **bridge: / region_name:** argument distinguishes which is happening.

model.add_role(name, type, goal) — defines one participant in a multi-role self-play system. MEANS: creates a named sub-model with its own architecture and objective inside a combined model (Section 14). Always paired with **.connect()** to wire roles together and **.set_evaluator()** to define success.

model.visualize() / **qai.studio(model)** — renders a live or static view of the model's internals. MEANS: **visualize()** is the general-purpose, technique-aware single view (Section 20); **qai.studio()** is the full multi-panel live experience (Section 93) built from several **visualize()**-style views at once.

model.weights.translate(vector) — converts raw weight numbers into a plain-language concept label. MEANS: correlates activation patterns with real inputs to produce a best-effort, confidence-scored guess at meaning (Section 78) — explicitly not guaranteed ground truth (Section 95's superposition caveat).

qai.dataset.split(data, ...) — divides a dataset into named subsets. MEANS: most commonly train/test, with **stratify_by** preserving class balance (Section 111). Produces a distinct **TestSet** type with no **train()** method, preventing data leakage by construction (Section 99).

result.diff(other_result) / **model.compare_logic(other)** / **qai.project_diff(...)** — comparison methods, each scoped differently: **result.diff()** compares two outputs; **compare_logic()** compares two logic implementations (Section 69); **project_diff()** compares an entire project's data+logic+weights+config together (Section 98).

`model.attach_skill(skill)` — activates a trained adapter capability on a base model. MEANS: wires a small, separately-trained module into the forward pass at its declared insertion point, without modifying the base model's own weights (Section 64).

`model.align(method, ...)` — a post-training refinement step specifically for behavior/values, not raw capability. MEANS: called after `train()`, alongside `finetune()`. ARGUMENTS: `method` (rlhf/dpo/constitutional/adversarial_training/debiasing/refusal_tuning, Section 99), plus method-specific data.

`model.knowledge_map()` — reports what the model does and doesn't know. MEANS: distinguishes a "weakness" (partial exposure, poor performance) from a "gap" (zero exposure) — the two need different fixes (Section 43).

`model.confidence(output)` — returns how certain the model is about a specific result. MEANS: a calibrated uncertainty estimate, not just a raw softmax number (Section 4) — used throughout logic blocks as a decision trigger (e.g. thinking loops, Section 75).

`model.swap_logic(new_logic)` — replaces a model's core `forward()` implementation with a different one. MEANS: changes what kind of thinking happens, distinct from changing weights (Section 68/81) — the mechanism behind self-architecting logic (Section 90).

`model.set_mutability("frozen" | "mutable")` — declares whether a model's weights can change outside of explicit `train()` / `finetune()` calls. MEANS: the safety gate for the whole self-modification system (Section 75) — defaults to `"frozen"`.

`model.think(task)` — runs an explicit reasoning pass before producing a final answer. MEANS: the plain method underlying `chain_of_thought()` (added this session) — exposes intermediate reasoning, usable in loops that revise an answer until confident (Section 75).

A note on scope, honestly stated: this covers the ~30 highest-frequency, most load-bearing methods — the ones nearly everything else in the document depends on understanding correctly. A genuinely complete audit of every method across all 115+ sections is a larger undertaking than one pass can honestly guarantee; if a specific method or argument elsewhere in

the doc is still unclear, flagging it directly remains the fastest way to get it properly documented next.

117. Daily-Use Developer Experience Features

Progress and feedback (highest priority – previously and notably absent):

```
model.train(data)
// → Epoch 3/50 [██████████░░░░░░░░░░] 47% | loss: 0.284 |
00:42 elapsed | ETA: 00:48
qai.dataset.stream(source).load(show_progress: true)
model.train(data, environment: slow_simulator)
// → "Training... still running (no ETA available) – last
activity: 3 seconds ago" // heartbeat, distinct from a
progress bar
```

Exploratory data analysis:

```
data.describe()  data.info()  data.correlation()
data.histogram("age")  data.value_counts("label")
```

Secrets/environment management:

```
qai.env.load(".env")    qai.env.get("HUGGINGFACE_TOKEN")
qai.env.require(["API_KEY", "DB_PASSWORD"])
```

Code quality & iteration:

```
qai fmt    qai lint    qai dev my_script.quantum // auto-
rerun on save
qai.timer("data_loading") { load_data() }
qai.profile(my_function)
qai env create "project_a"    qai env activate "project_a"
// dependency isolation, distinct from the lockfile (Section
42)
```

CLI daily-use quality of life:

```

model.accuracy().copy_to_clipboard()
result.as_markdown_table()
model.train(data, dry_run: true)  model.export(path,
dry_run: true)
qai undo
qai calc "512 * 4 / 8"  qai shape Tensor<[32,512,768]>
qai t / qai r / qai ls  // muscle-memory aliases
model.trian(data)  // → "did you mean 'train'?"
qai history  qai last
model.train(data)  // auto-prints: "✓ Training complete.
Accuracy: 0.91. Time: 4m 12s. Saved to: my_model.qmodel"
qai repl  // up-arrow history, Ctrl+R search
qai explain-error --from-clipboard
qai catch_me_up  // summary of what changed since last
session
qai validate my_training_script.quantum  // checks
schema/paths/keys BEFORE a long run starts
qai train --help=epochs  // inline flag explanation
qai train my_model.quantum && qai notify "training finished"
qai init my_project  // auto-generates a starter README,
kept updated

```

Priority notes: progress bars/ETA were a genuine, notable gap for a tool this comprehensive; EDA and `.env` handling are close behind in daily importance; "did you mean" typo suggestions, the auto-printed training summary, the running/hung heartbeat, and pre-run config validation are the standout items among the rest — each addresses a real, frequent point of friction or wasted time, not just cosmetic polish.

118. Context vs. Memory — A Real, Necessary Distinction

Two genuinely different things, often conflated:

- **Context:** the finite window of tokens visible to a model in one call — expensive, and everything in it costs real time/money on every single turn, whether or not it's still relevant.
- **Memory:** information that persists BEYOND the current context, stored separately, and pulled back in only when actually relevant.

A system with no real distinction between these two degrades over time exactly the way a long conversation does without it: context keeps accumulating, most of it stale, and an increasing share of every response's real cost goes toward re-reading things that no longer matter, rather than doing the actual task.

Real, established techniques, formalized as concrete mechanisms:

```
qai.context.sliding_window(max_tokens: 8000)
// simplest: drop the oldest turns once the limit is hit.
// Fast, but can genuinely lose
// something important with no warning.

qai.context.compact(strategy: "summarize")
// compress old turns into a shorter summary instead of
// dropping them outright –
// preserves the gist, sheds the raw token cost.

qai.memory.retrieval_store(key: str, value: any,
  relevance_check: fn)
// the stronger fix: persist information in a SEPARATE store
// (file/db/vector index),
// and only pull the RELEVANT pieces back into context per-
// turn – not everything, always.

qai.memory.task_scoped_forget(on: "task_complete")
// once a task finishes, collapse its full working detail
// down to just the outcome
// ("searched X, found Y, used it for Z") rather than keeping
// the entire transcript live.
```

Logic-driven memory management — a real, natural extension of `logic` {}, not a separate system:

```
model.logic {
  input(current_context)
  let decision = model.think("what here is still relevant
  to finish this task, and what is safe to forget?")
  qai.memory.write(decision.keep)
  qai.memory.discard(decision.drop)
  output(decision.keep)
}
// the model's OWN reasoning decides what's worth persisting,
```

```

rather than a fixed,
// dumb rule (pure sliding window) or a fixed schedule
(compact every N turns) --
// this is how real, more advanced agentic memory systems are
actually built today.

```

Inspecting and moving things between context and memory – the missing piece, added here:

```

qai.context.inspect()
// → a real, readable table: [{turn: 4, summary: "user asked
about X", tokens: 340, still_relevant: true}, ...]
// -- the .weights.as_table() equivalent for context, not
just an opaque blob

qai.context.demote(item)
// move something OUT of the live context window and INTO
persistent memory --
// it stops costing tokens on every turn, but isn't lost,
just archived

qai.context.promote(memory_key)
// pull something back FROM memory INTO the live context --
e.g. the user
// directly asks about something that was archived earlier

model.logic {
  input(current_context)
  for item in qai.context.inspect() {
    if not item.still_relevant {
      qai.context.demote(item)          // real,
conditional, logic-driven decision
    }
  }
  if user_message.mentions(qai.memory.keys()) {
    qai.context.promote(matching_key)    // bring back
exactly what's being asked about, nothing else
  }
  output(qai.context.inspect())
}

```

This closes the actual gap: `logic {}` can now write real `if` statements against what's *currently* in context (not just memory), and move information

both directions — demoting what's gone stale, promoting exactly what's asked about back in, instead of either keeping everything or discarding everything.

119. Verify-Before-Predict — Tool-Augmented Reasoning as a Logic Pattern

A model's default mode is prediction — producing the statistically likely next answer from learned patterns, without actually checking whether it's true. For anything checkable (math, code correctness, a factual claim with a real source), this is a real, avoidable source of error. The fix: let logic force a genuine verification step before committing to an output, rather than trusting the first guess.

```
model.logic {
  input(problem)
  let guess = model.predict(problem)

  if problem.is_verifiable {
    let verification_code = model.generate_code(to_check:
guess)
    let real_result =
qai.connect_vm().run(verification_code)

    if real_result != guess {
      output(real_result) // trust the ACTUALLY
COMPUTED answer, not the guess
    } else {
      output(guess)
    }
  } else {
    output(guess) // no real way to verify a
subjective/creative answer -- predict honestly
  }
}
```

This is the same real mechanism behind tool-augmented/program-aided reasoning in production systems today: don't just predict a math answer from pattern — write real code, actually run it, and report the computed

result instead of the guess. The honest limitation: this only works for genuinely *verifiable* claims (math, code, anything with a checkable ground truth) — a creative or subjective answer has nothing to verify against, and logic should recognize that distinction rather than pretending everything is checkable.

120. Method Glossary — Completeness Pass (Session Methods)

Every method mentioned in conversation since Section 119, given a real definition — following the same standing rule as Section 116: nothing stays as a bare example with no meaning.

Consolidation & safety:

- `model consolidate(fact, confidence_threshold)` — promotes something from a named memory store permanently into weights via a small, targeted finetune (not full retraining). Only proceeds above the confidence threshold, since baking a wrong guess into permanent weights is worse than a wrong guess in temporary memory.
- `model.weights_diff(before, after)` — treats the weight table like a database you can diff; returns exactly which weights changed and by how much after any training/finetune/consolidation event.
- `model.capability_regression_check(old_tests)` — reruns a fixed set of previously-passing examples after any weight change; flags any that now fail. The concrete defense against catastrophic forgetting.
- `model.set_external_anchor(verified_test_set)` — a small, fixed, human-verified test set that self-play/self-testing loops can read from but never modify or train on. Prevents model collapse: a loop judging only itself has no way to detect quality drift.
- `model.evaluate(against=anchor)` — scores a model against the external anchor specifically, as distinct from `self_test()`, which uses the model's own generated/judged tests.
- `model.provenance()` → `{sources, licenses, collected_via}` — honest metadata on what a model was trained on; reports origin, not a judgment on data quality.

Memory system (Section 118, extended):

- `model.memory.attach(name, type)` — attaches a named, independent memory store (file/vector_store/key_value/ephemeral); a model can hold any number of these, purpose-named by the user.
- `model.memory.list()` — returns the names of all currently attached stores.
- `model.memory[name].write(x)` / `.retrieve(query)` / `.confidence(x)` — per-store read/write/confidence-check.
- `model.memory.access_control(name, writable_by)` — restricts which logic blocks may write to a given store (e.g. training logic can write; runtime logic can only read).
- `model.memory[name].rollback(to_version)` — per-store versioning; reverts a single memory store to an earlier state, independent of weight snapshots.

Context (Section 118, extended):

- `gai.context.inspect()` / `.promote(key)` / `.demote(item)` — already defined in Section 118; repeated here only to confirm no separate meaning was intended.

Hardware-awareness (ties to existing Section 100 methods, not new primitives):

- `gai.power.level()` — reuses the existing hardware-awareness system; returns current power/battery level for a logic condition to act on.
- `gai.memory.ram_usage()` — current RAM pressure, same system.
- `model.reduce_activity()` / `model.pause_training()` — logic-triggered responses to low power/high RAM; not new mechanisms, just named entry points into the existing budget/resource system.

Continuous operation:

- `model.think_ahead(context)` — a real, named pattern (not a new primitive): the model reasons proactively on the current context without new user input, inside a non-terminating `loop {}`. Confirmed in the last exchange to be a genuinely new idea from this session, not something previously documented.

- `qai.dynamic_interval()` — returns a variable wait time for a continuous loop, adjustable by logic (e.g. slower when power is low).

Human oversight:

- `qai.human_review.request(question, answer)` — pauses logic execution, surfaces a real decision to a human, waits for a true/false or approve/reject response before continuing.
- `model.flag_for_retrain(question, answer)` — marks a specific input/output pair for inclusion in the next retraining pass, typically set from a failed human review.

Multi-model training (real, not illustrative — the two-model ask/verify/answer pattern itself is NOT documented as a worked example per the user's request, but the underlying mechanism it uses is real and already covered): `model.add_role()`, `model.self_play()` — Section 14.
`qai.web.find_sources()`, `Dataset.from_sources()` — Section 12/97's real data-sourcing methods, renamed here for clarity.

121. Batteries-Included Standard Library

A real design principle, not just a list: Quantum ships small, common utilities built in, so ordinary automation never requires an external import — only genuinely specialized, heavy capability does.

Built in by default (no import needed):

```
qai.datetime.now() / .format() / .diff()
qai.power.level() / qai.hardware.ram_usage() / .disk_usage()
qai.file.read() / .write() / .exists()           // basic I/O
qai.env.load() / .get() / .require()             // Section
117's secrets handling
qai.math.*                                       // basic
numeric operations
qai.random.seed() / .choice() / .shuffle()
```

Requires an explicit import (specialized, heavy, or third-party):

```
import advanced_stats      // e.g. a full statistics package
import some_vision_library // e.g. specialized image
```

processing beyond what a Type needs

The dividing line: if it's small, common, and nearly every project needs it, it's built in. If it's heavy, specialized, or genuinely optional, it's an explicit import — keeping the built-in surface lean rather than importing everything by default.

122. GUI/IDE — A Real, Later-Phase Goal (Not Designed In Depth Yet)

Explicitly scoped as a phase-3 goal, deliberately not over-designed now: a visual development environment for Quantum, similar in spirit to a game engine's editor or a database GUI (e.g. MySQL Workbench), but purpose-built for model development.

Rough shape, kept intentionally light:

- A scripting console/IDE pane for writing real `logic{}` and Type code directly (an "IDLE for models")
 - Drag-and-drop panels for things that don't need code: attaching a memory store, wiring `pipe_to()` /ensemble connections, adjusting hyperparameters
 - Some things scriptable, some things point-and-click — not an either/or, both available depending on the task
 - Depends entirely on the underlying language/CLI being solid first — this is explicitly NOT something to design in detail before the core is proven, per the earlier honest assessment in Section 115
-

Project Notes

- Currently a personal project/learning tool; may be shared with a small group of friends later if it proves useful
- Core differentiator vs Python/PyTorch/TensorFlow: everything above as language-native primitives instead of assembled from 5+ separate

libraries, with interpretability and non-hardcoded extensibility built in from the start rather than bolted on